Memory Hierarchy

Main Points

- Memory technologies (just a quick overview)
- Memory hierarchy
- Cache memories
- Measuring and Improving Cache Performance

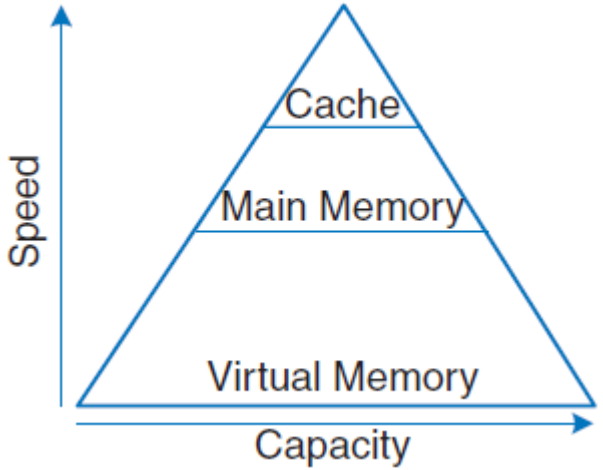
Credits: several figures in these slides come from “Computer Organization and Design. The Hardware/Software Interface” by D.A.Patterson, J.L. Hennessy, 5th edition.

Memory Technologies

- Different memory technologies
- Volatile memories:
  - Latches, flip-flops, register files (or simply ***registers***)
  - SRAM (Static Random-Access Memory)
  - DRAM (Dynamic Random-Access Memory)
- Non-volatile memories:
  - ROM
  - NVRAM
  - Flash memory
  - Magnetic disks
  - Optical disks
  - Tape
  - …

Cost vs Capacity vs Access Time

- **SRAM:**
  - Access Time (ns): 0.5 - 1 Bandwidth (GB/s): 25+
  - Price ($/GB): 5,000
  - Used for registers and caches
- **DRAM**
  - Access Time (ns): 10 - 50 Bandwidth (GB/s): 10
  - Price ($/GB): 7
  - Used for: RAM
- **Flash memory**:
  - Access Time (ns): 20000 (20 us) Bandwidth (GB/s): 0.5
  - Price ($/GB): 0.40
  - Used for: SSD disk (secondary and virtual memory – non volatile)
- **Magnetic disks:**
  - Access Time (ns): 5000000 (5 ms) Bandwidth (GB/s): 0.75
  - Price ($/GB): 0.05
  - Used for: HDD disk (secondary/tertiary storage – non volatile)



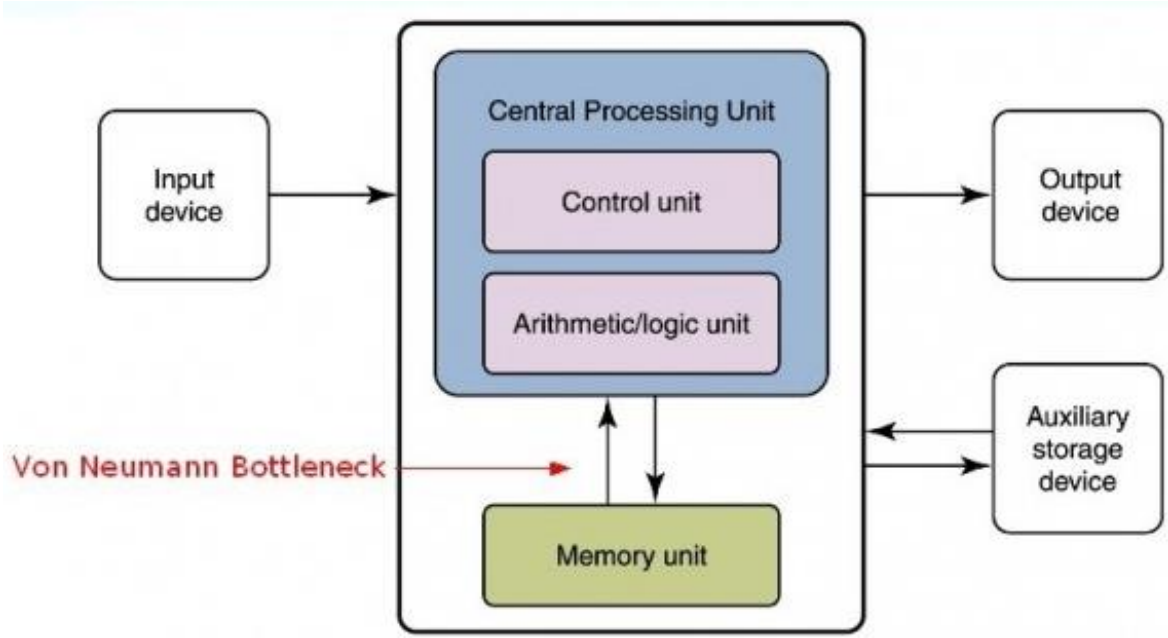
(*) Typical values for 2015

Contrasting needs

- *General rules:*
  - Larger memory is slower and cheaper
  - Smaller memory is faster and more costly

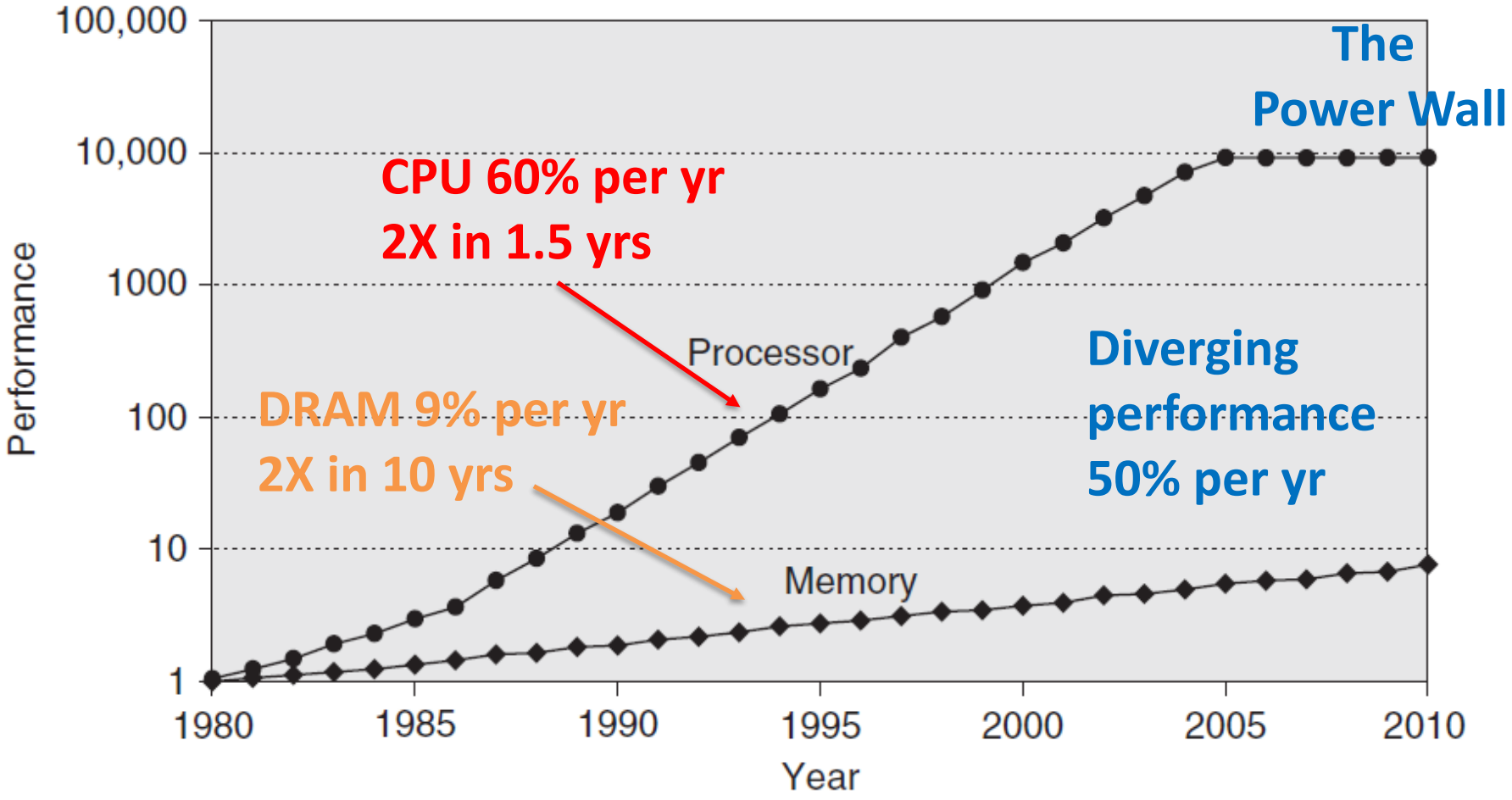
- Programmers/users need large and fast memories
  - Large enough to fit all needed data
  - Fast to mitigate the *von Neumann bottleneck*

von Neumann architecture

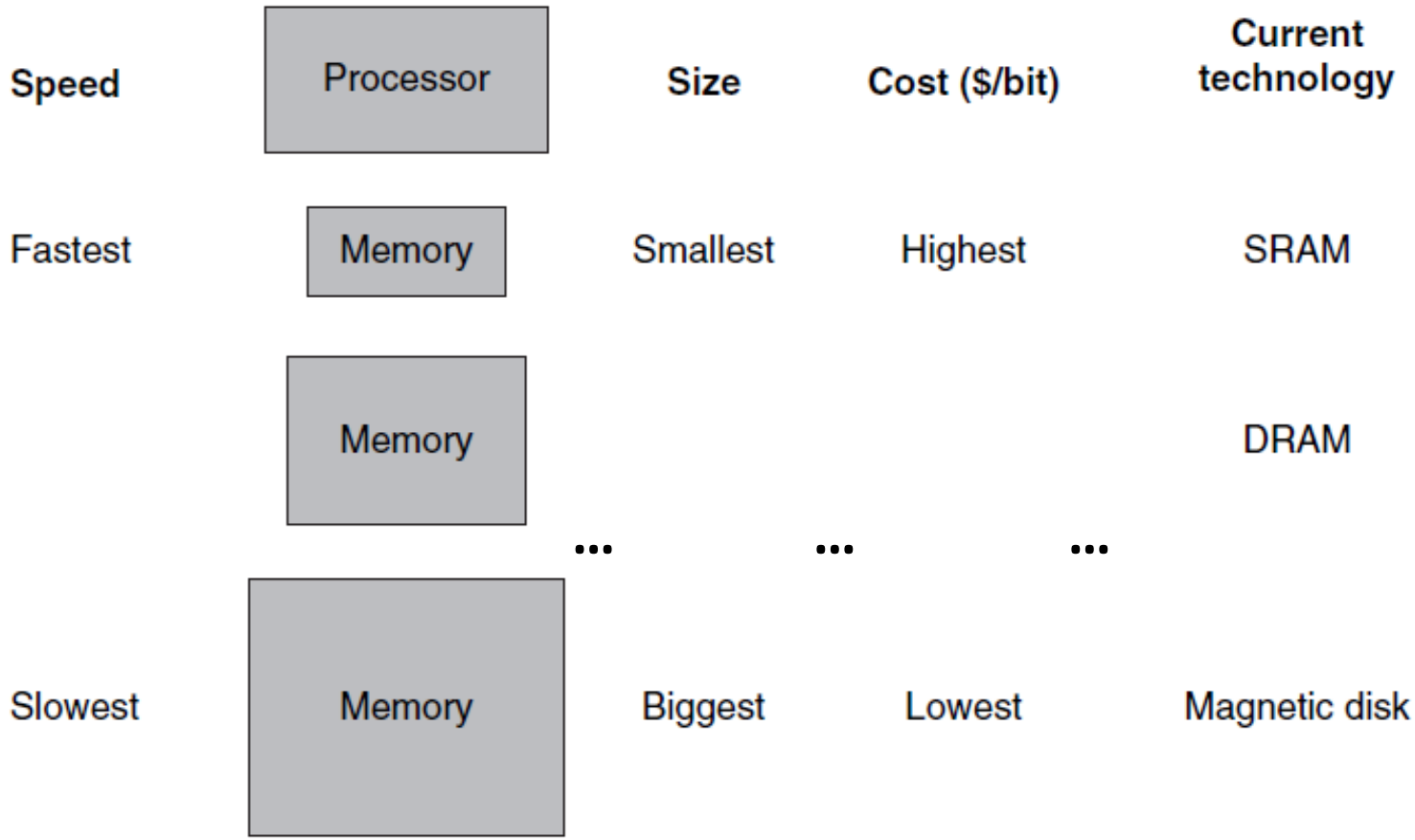


- Computer system performance is limited due to the relative speed of CPUs compared to top rates of data transfer between off-chip memories and CPUs.

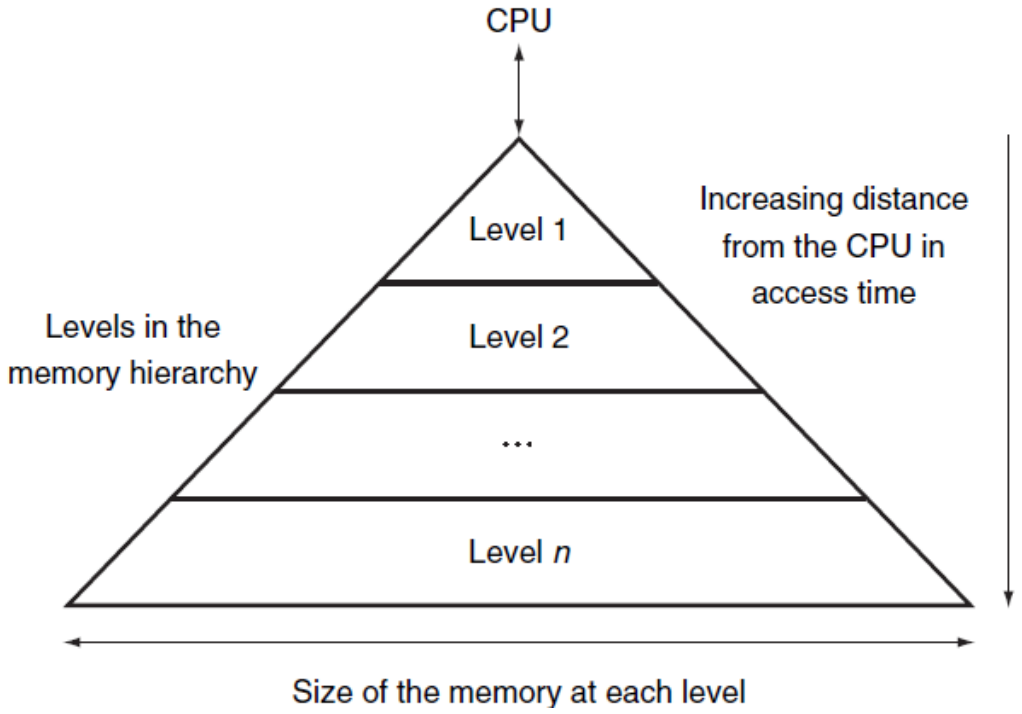
Von Newmann bottleneck



Basic structure of a memory hierarchy



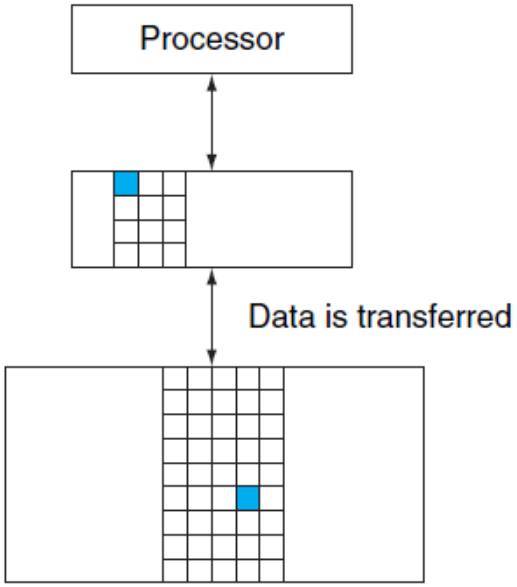
Goal of the memory hierarchy



Goal: By implementing the memory system as a hierarchy, the programmer has the *illusion* of a memory that is as large as the largest level of the hierarchy and (*almost*) as fast as the closest level.

Illusion of large and fast memory

- At the beginning, data resides in the last level of memory
- Referred data moves from level n to level n-1 using different data transfer granularity (e.g., pages, blocks)
- Data movements is transparent to the high-level programmer
- Level n-1 contains a subset of data present in level n (***inclusive*** memory levels)



Questions

- What if the referred data is already present in one memory level ?
  - We need a mechanism to find the right data
- What if one memory level does not have the data and it is full ?
  - We need both mechanisms and policies to replace one of data present to make space for the new one

Terminology

- If the data requested by the processor appears in some block in the closest memory level, this is called a hit. Otherwise, the request is called a miss and the next memory level is then accessed to retrieve the block containing the requested data.
- The hit rate (frequency of successes) is the fraction of memory accesses found in the upper level (i.e., closer to CPU)
  - used as a measure of the performance of the hierarchy
- The miss rate is the fraction of memory accesses not found in the upper level
- The miss penalty is the time to replace a block in level n with the corresponding block from level n-1.
- The miss time is the time to obtain the element in case of miss
  - miss time = miss penalty + hit time

Hit rate, miss rates and *AMAT*

$$MR = \frac{Number\ of\ misses}{Number\ of\ total\ memory\ accesses} = 1 - \text{HR}$$

$$HR = \frac{Number\ of\ hits}{Number\ of\ total\ memory\ accesses} = 1 - \text{MR}$$

$$AMAT = \underbrace{t_{M0}}_{\text{hit time}} + \underbrace{MR_{M0}}_{\text{miss rate}} * \underbrace{(t_{M1} + MR_{M1} * (t_{M2} + MR_{M2} * (t_{M3} + \ldots)))}_{\text{miss penalty}}$$

- ***AMAT*** is the ***Average Memory Access Time***
  - hit-time + (1 – hit-rate)*miss-penalty
- MRx is the Miss Rate (probability of miss) at memory level x
- Tx is the access time for a memory hit at level x (hit time)

Observation: If the hit rate is high enough, the memory hierarchy has an effective access time close to that of the highest (and fastest) level and a size equal to that of the lowest (and largest) level.

The locality principle

- *Locality of reference* (or ***locality principle***) refers to the phenomena in which a program tends to access *the same set of memory locations for a given time period*.
- It has been observed that, If the program refers a memory location, then
  - *The same memory location* will be used again soon with a high probability
  - The elements “close to” the memory location just accessed will be referenced soon with a high probability

The locality principle

- The *locality principle* is the ***driving force*** that makes the memory hierarchy work properly
- It increases the probability of reusing data blocks that were previously moved from level n to level n-1 (i.e., closer to the CPU), thus reducing the *miss rate*.

Locality characterization

- **Temporal locality** (or *data reuse*): data items referenced recently are likely to be referenced again soon
  - Examples: instructions in a loop, data variables

```
for(int i=0; i<10;++i) { s1 += i; s2 -= i; }
```

  - *Memory hierarchies take advantage of data reuse* ***by keeping more recently accessed data items closer to the CPU***

Locality characterization

- **Spatial locality**: data items near those referenced recently are likely to be referenced again soon
  - Examples: sequential instruction accesses, data elements of an array

```
for(int i=0;i<10; ++i) func(A[i]);
```

  - *Memory hierarchies take advantage of spatial locality* ***by moving data items in blocks*** *(i.e., contiguous words in memory) from level n to level n-1*

Data transfering

- Data is transferred between only two adjacent memory levels at a time.
  - The processor accesses only at the first level
- Transfers always happen at ***block*** granularity between memory levels to exploit spatial locality
  - Memory word granularity between level 0 and registers
- The block size may vary among levels
  - For the cache, the block is called *cache line* or *cache block* (typical values are 64 - 128 bytes – 8, 16 memory words)
  - Pages or segments for the RAM
  - Disk block for the disk

Locality examples

- Let's consider the following snippet of C code:

```
// sum and A are global variables
int i;
for(i=0, sum=0;i<N;++i)
  sum += A[i]
```

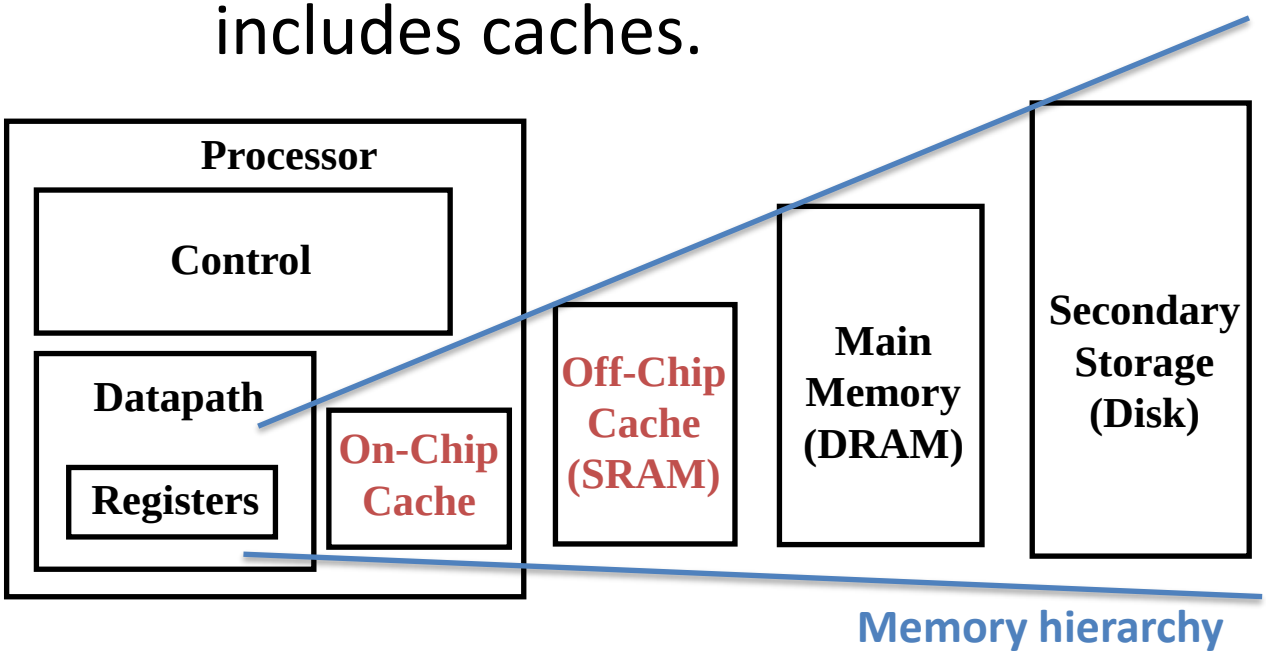
one possible compilation

```
 @ r0=&A, r1=&sum, r2=N, r3=i
loop:  cmp r3, r2
       beq end
       ldr r12, [r0, r3, lsl #2]
       ldr r4, [r1]
       add r4, r4, r12
       str r4, [r1]
       add r3, r3, #1
       b loop
end: …
```

- Loop body executed N times; each **instruction** fetched N times in sequence → *both temporal and spatial locality for the instructions*
- *sum* is repeatedly read and written over time (the same for the loop index i, but it is usually kept in a register) → *sum* exhibits temporal locality
- *A* is stored contiguously in memory; *A* exhibits spatial locality because its elements are accessed one after the others
  - A[0], A[1], …. A[i-1], A[i], A[i+1], A[i+2], …. A[N-1]

Cache memories

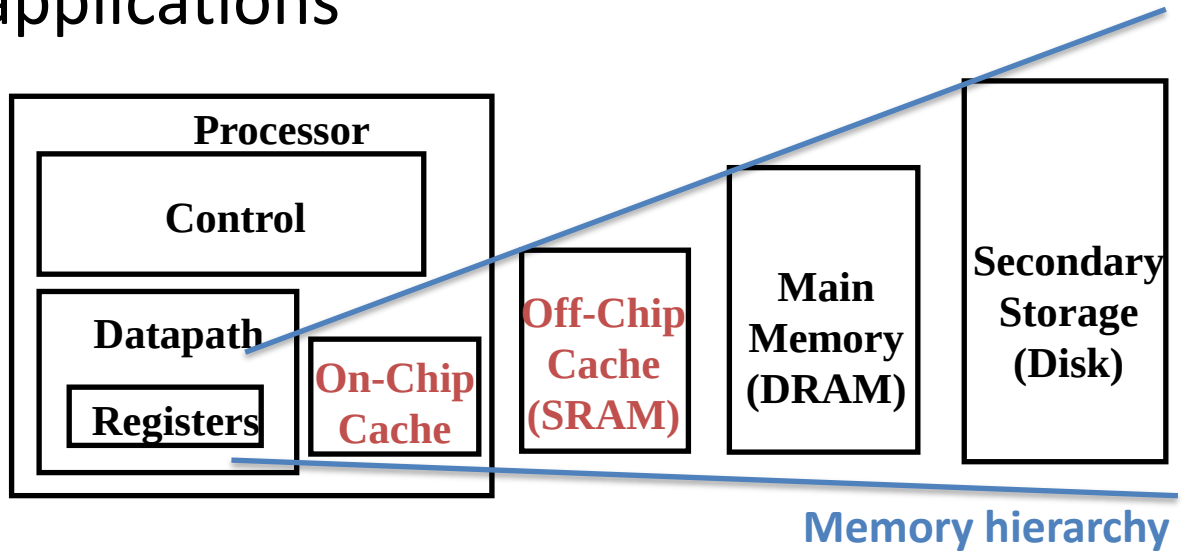
- Cache memory is the level of the memory hierarchy closest to the CPU
  - Every general-purpose computer built today, from servers to low-power embedded processors, includes caches.



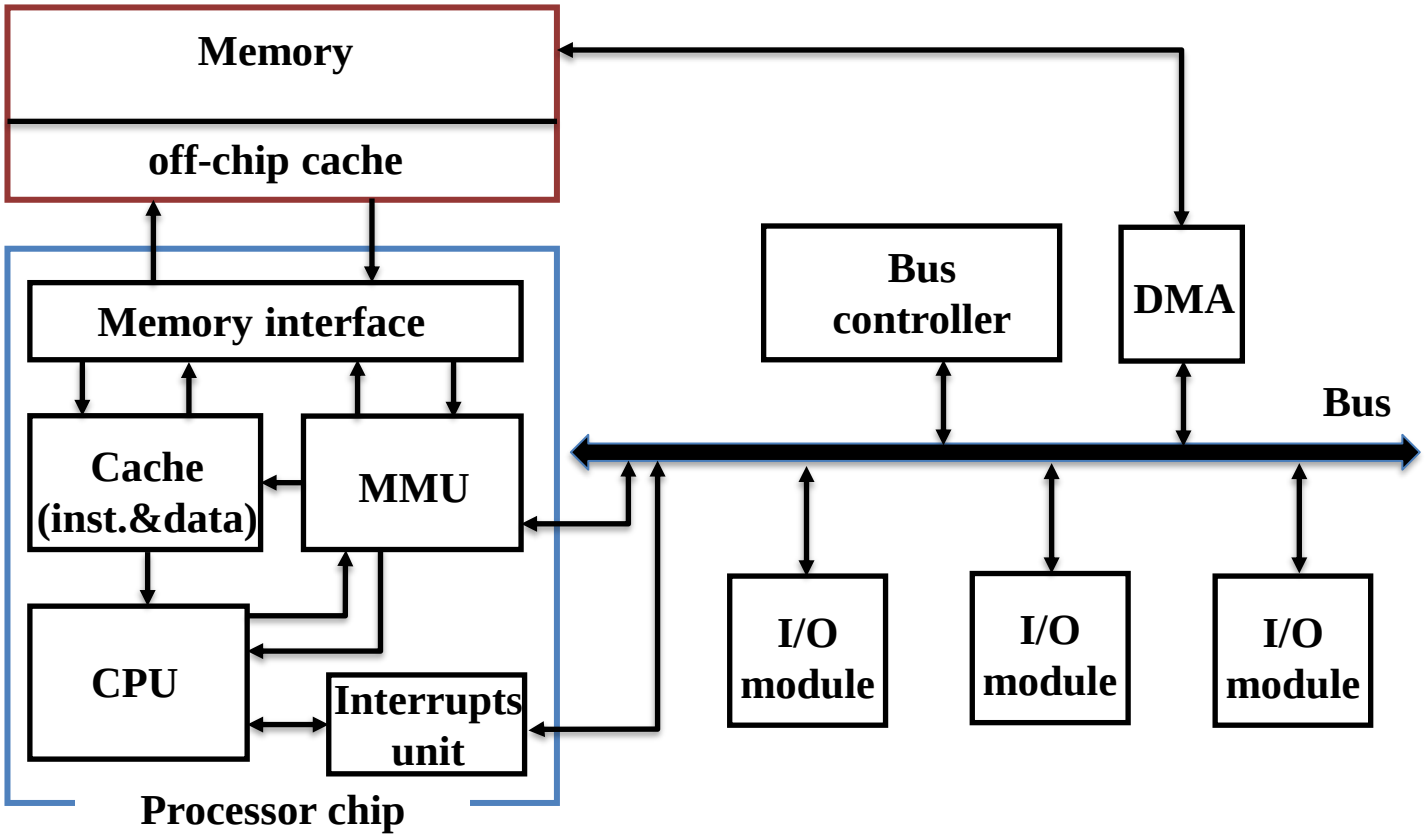
Cache: a hiding place especially for concealing and preserving provisions or implements.
– Merriam Webster Online Dictionary, 2015. www.merriam-webster.com

Handling data movements

- Between first level cache and registers handled by the compiler (ldr*, str* instructions)
- Between cache levels and between caches and RAM handled by the HW (microarchitecture)
- Between storage devices and RAM handled by the OS and applications

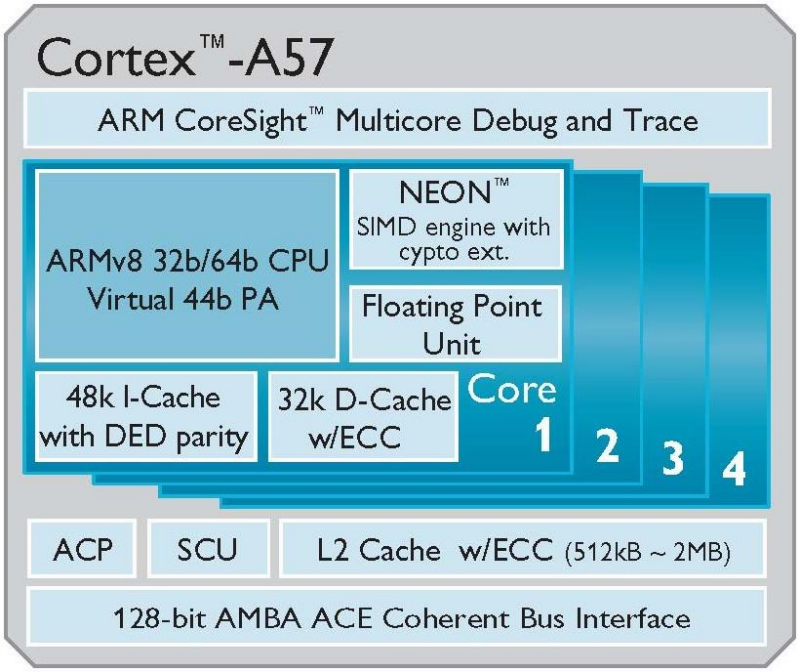


Cache-based architecture (logical scheme)

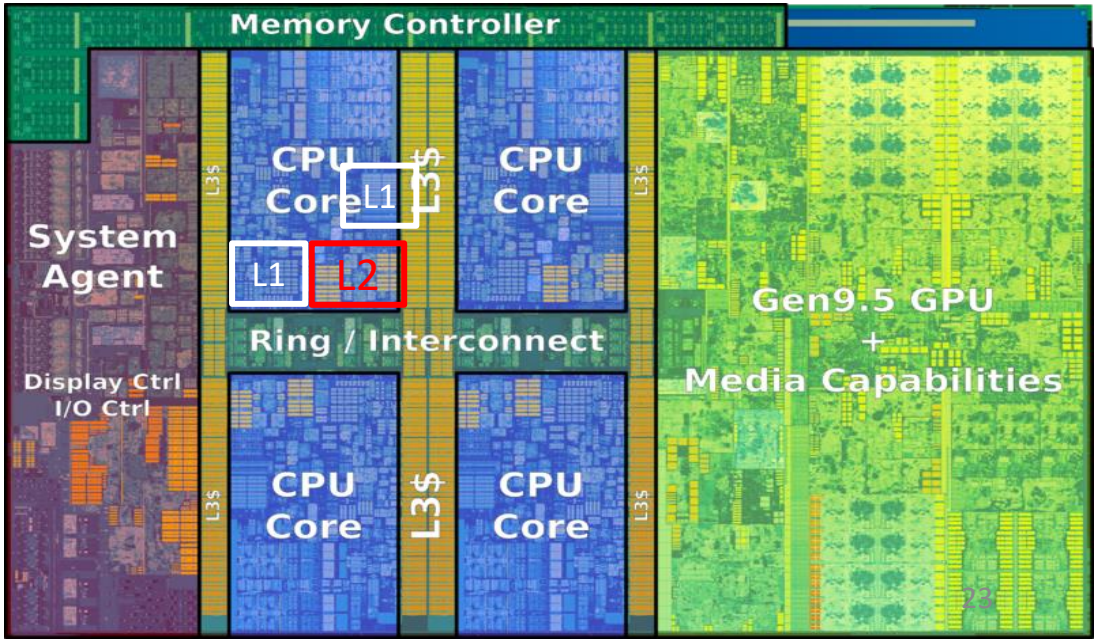


Caches on modern processors

- Multi-levels on-chip caches (L1, L2, L3)
- Per-core private cache (L1)
- Large on-chip shared cache (L2 or L3)
- *Separate data and instruction caches* (e.g., L1d, L1i)



Intel Keby Lake



Caches usage

- Caches are organized in *lines*, each line contains a block of memory words (e.g., 8 - 16 memory words)
- The first time the processor asks for a memory word a ***cache miss*** occurs
  - The block containing the memory word is transferred into the cache
- Next requests
  - If the data is present in the block → **cache hit**
  - If the data is not present → **cache miss**
    - The block containing the data is transferred into a cache line

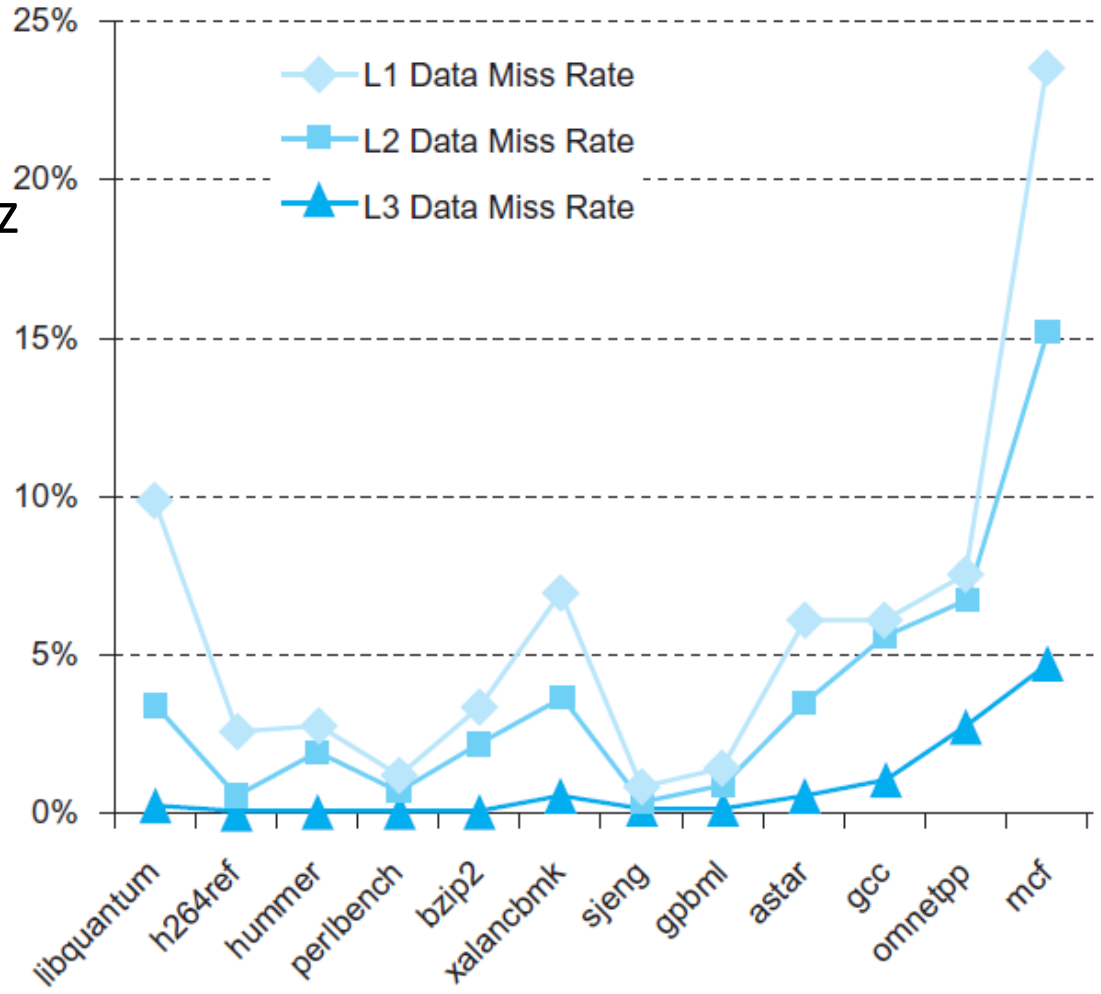
Cache effects on *AMAT*

- The usage of large caches in a memory hierarchy helps to reduce the von Neumann bottleneck
- Quantitative example. Suppose the following values:
  - $t_M = 50ns$ (main memory service time)
  - $t_{L1} = 1ns$ (L1 hit time, i.e., cache hit service time)
  - Miss rates (MR_{L1}): 5%

1. No cache: *AMAT*= **50ns**
2. With L1 cache: *AMAT*= $t_{L1} + MR_{L1} * \ t_M$ = 1+0.05*50= **3.5ns**

Cache miss rates

- SPECCPU2006 benchmarks
- Intel Core i7 920 @2.66GHz
  - 4 cores, 8 HW contexts
  - L1d 4x32KB, L1i 4x32KB
  - L2 4x256KB
  - L3 8MB shared
- t_M about 100 cycles
- t_{L1} 4 cycles
- t_{L2} 10 cycles
- t_{L3} 35 cycles



Cache Performance

$$CPU_{time} = ClockCycles * ClockCycleTime = IC * CPI * ClockCycleTime$$

where:

- IC (*Instruction Count*) is the number of program instructions executed
- CPI (*ClockCycles Per Instruction*) is defined as $\frac{ClockCycles}{IC}$

- CPU time can be further divided into the cycles that the CPU spends executing the instructions with no misses ($CPI_{Perfect}$) and the clock cycles that the CPU spends waiting for the memory system (CPI_{Stall} *or Memory-stall ClockCycles*)

$$CPI = (CPI_{Perfect} + CPI_{Stall})$$

- The *Memory-stall clock cycles* can be defined as the sum of the stall cycles coming from reads and writes. For simplicity, let's assume that the read/write miss rates and miss penalties are the same for load and stores
  - Here we also assume that ***write buffer*** stalls are negligible (see "Optimizing Writes")

$$CPI_{Stall} = \underbrace{\frac{Memory\ Instructions}{Program\ Instructions} * Miss\ rate}_{\text{Miss rate per memory instruction}} * Miss\ penalty$$

Cache Performance Example

- Assume a miss rate of 2% for the instruction cache and of 4% for the data cache, a miss penalty of 100 cycles for all misses, and a frequency of 36% of loads and stores. If the CPI is 2 without memory stalls (i.e., $CPI_{Perfect}$), determine how much faster the processor runs with a perfect cache that never misses.

- CPI memory stalls for instructions and data accesses:

  $CPI_{Stall-Instr} = 1 * 0.02 * 100 = 2$ cycles

  $CPI_{Stall-Data} = 0.36 * 0.04 * 100 = 1.44$ cycles

  $CPI_{Stall} = 2 + 1.44 = 3.44$

  Therefore, the total CPI including memory stalls is: $CPI = 2 + 3.44 = 5.44$

$$\frac{CPU_{time\ with\ stalls}}{CPU_{time\ perfect}} = \frac{IC * (CPI_{Perfect} + CPI_{Stall}) * ClockCycleTime}{IC * CPI_{Perfect} * ClockCycleTime} = \frac{5.44}{2}$$

- The performance with the perfect cache is better by a factor of 2.72

Designing a caching system

- **Main objective**: minimize *AMAT*

  *AMAT = hit-time + miss-rate * miss-penalty*

- For minimizing AMAT we can:
  - Reduce the miss rate
  - Reduce the miss penalty
  - Reduce the hit time

Designing a caching system

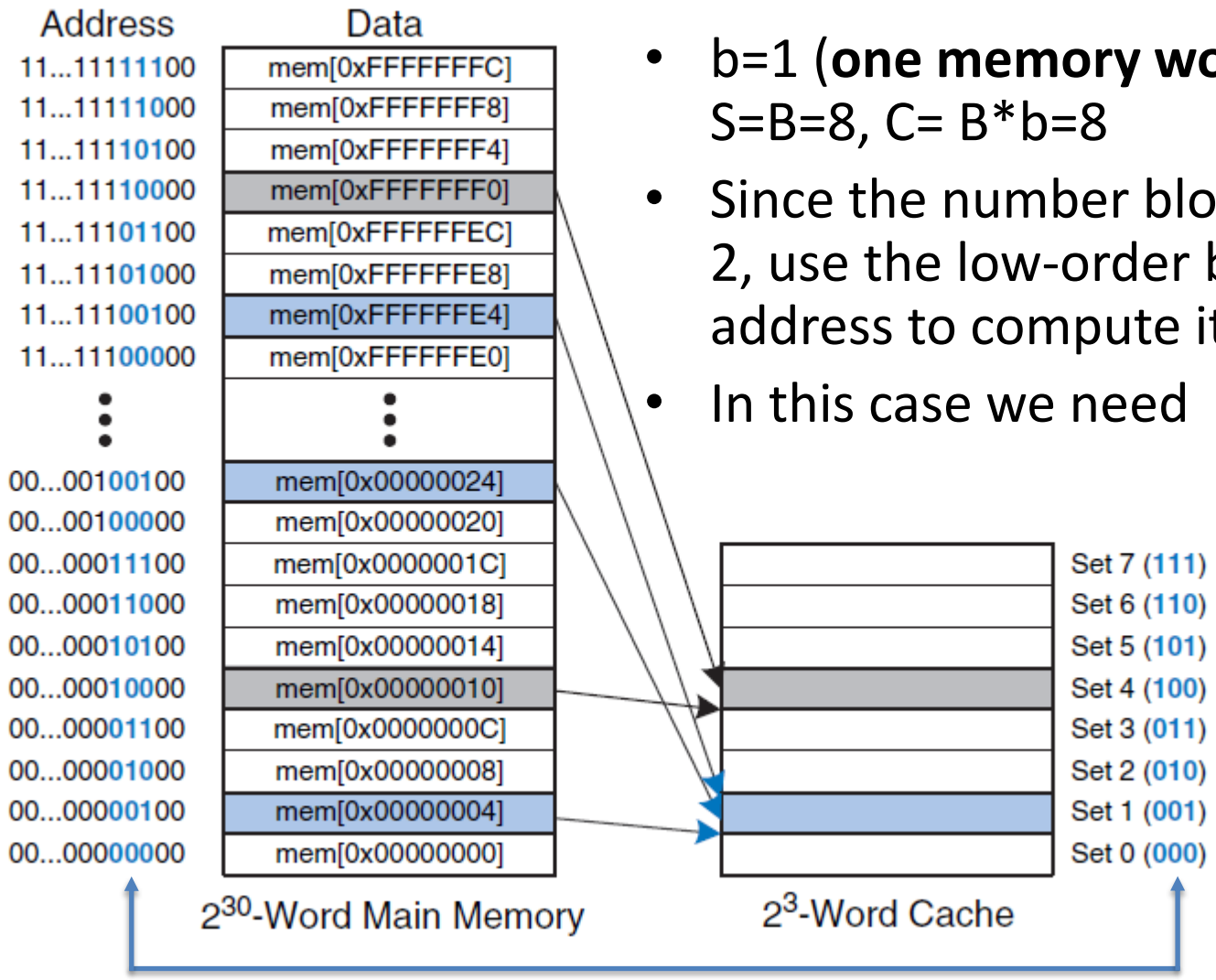
- Questions:
  - How large is the cache block? How many blocks for each level?
  - How do we know if a data item is present in the cache?
    - We need a mapping function: memory address -> cache block id
  - If the data item is not present, how to retrieve the block?
  - How to deal with block conflicts?
  - Which is the replacement policy?
  - What happen if the block to be replaced has been modified?
  - How to keep consistent different memory levels?
    - What does it mean data consistency?

How is data found?

- A cache of capacity ***C*** with ***B*** blocks (or *lines*) is organized into ***S*** sets each one holding some blocks. ***b*** is the number of words per block.
- Example b=4

- The ***mapping function*** maps one memory address to exactly one set
- Caches can be categorized based on the number of blocks in a set
  - *Direct mapped* cache has #S=#B
  - *N-way set-associative* cache. Each set contains *N* blocks. *S= B/N*
  - *Fully associative* cache has S=1
- From now on, let's consider a system with 32-bit addresses and 32-bit words → the memory has 2^{30} words.

Direct Mapped Cache



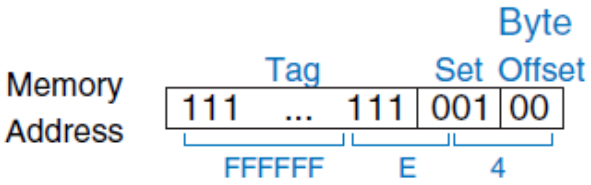
- b=1 (**one memory word per block!**), S=B=8, C= B*b=8 (**)
- Since the number blocks is a power of 2, use the low-order bits of a block address to compute its cache location
- In this case we need $log_2 8 = 3\ bits$

** This is just an illustrative example, do not exist caches with b=1!!!

Direct Mapped Cache

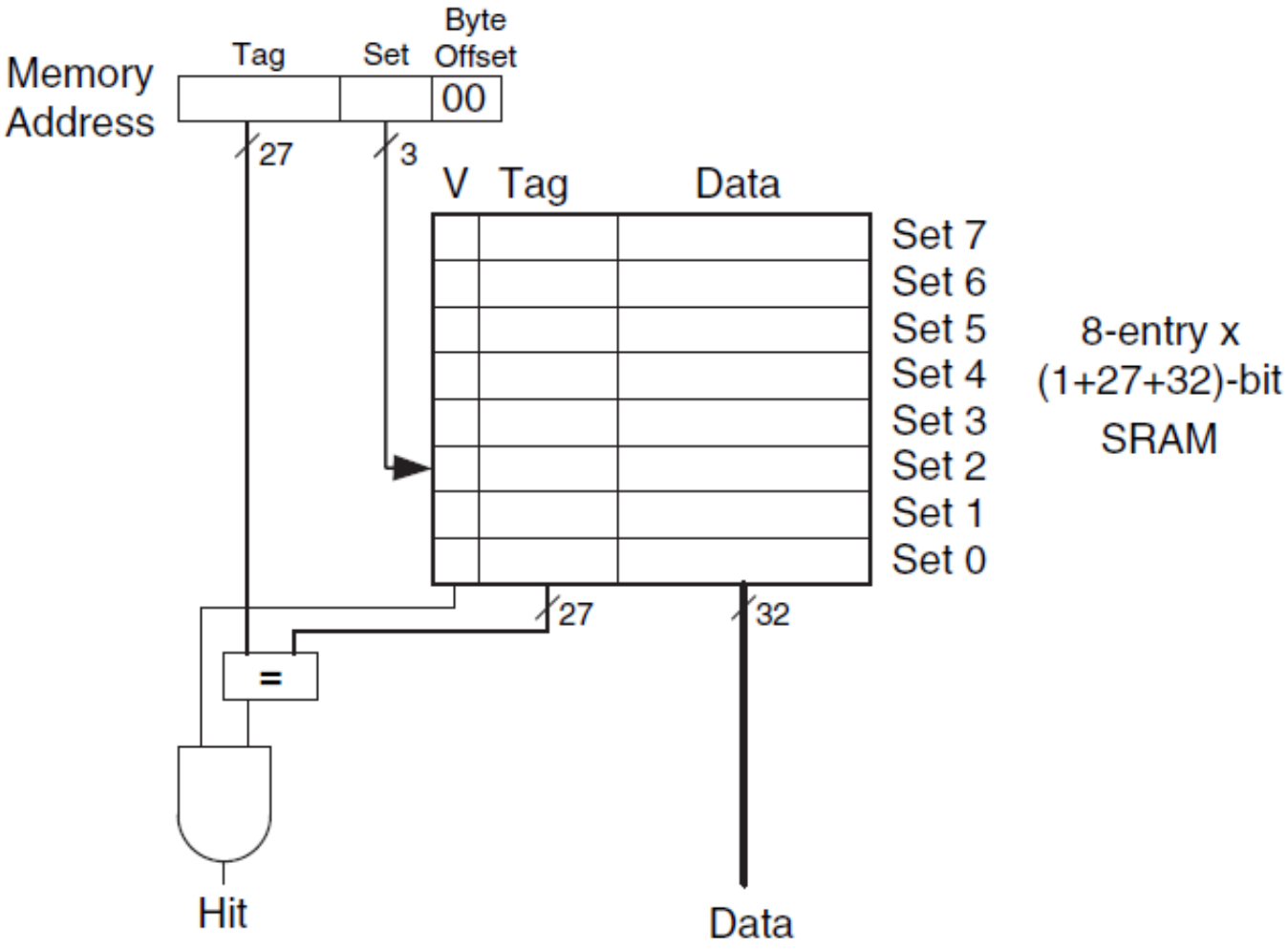
- How do we know which block is in a cache location?
  - We need a set of **tags to each cache location** identifying the block address in cache
  - Actually, the tag only needs to contain the higher-order bits of the memory address

Address 0xFFFFFFE4



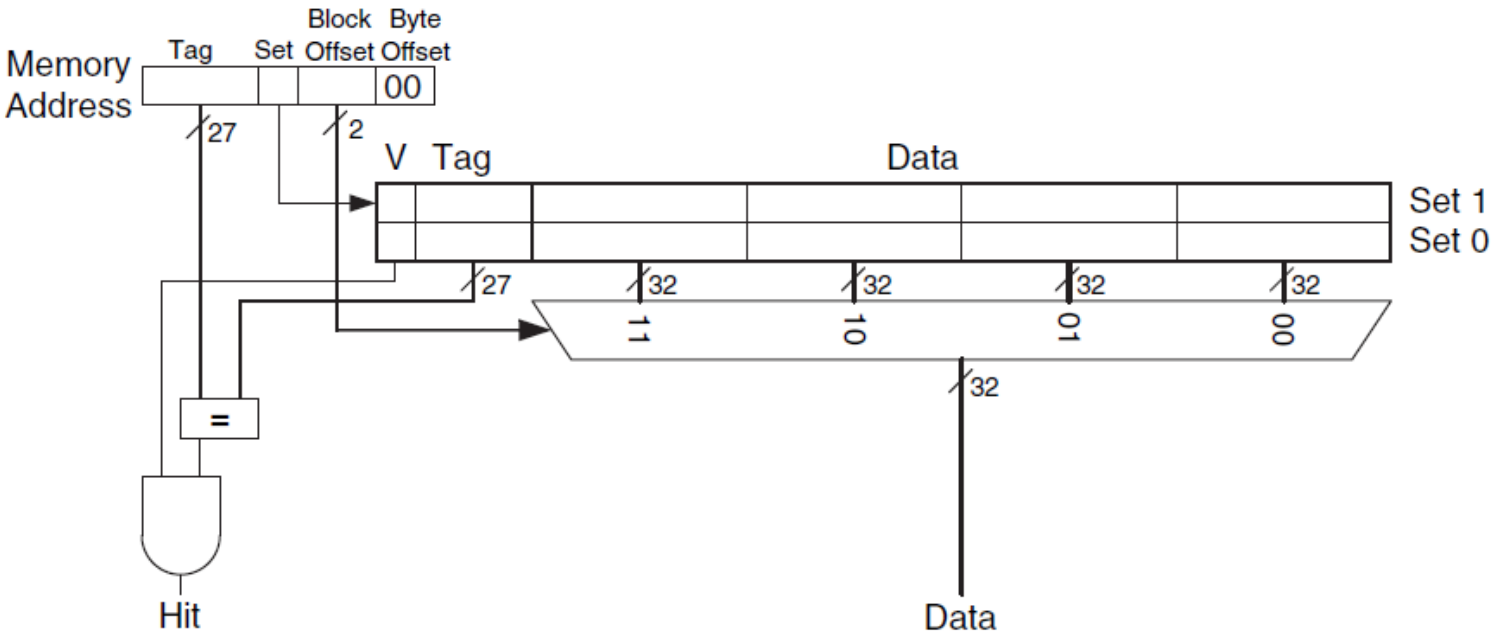
- How do we know whether the data in a cache location is valid?
  - We need **a valid bit to each cache location** (valid=1; invalid=0)

Direct Mapped Cache

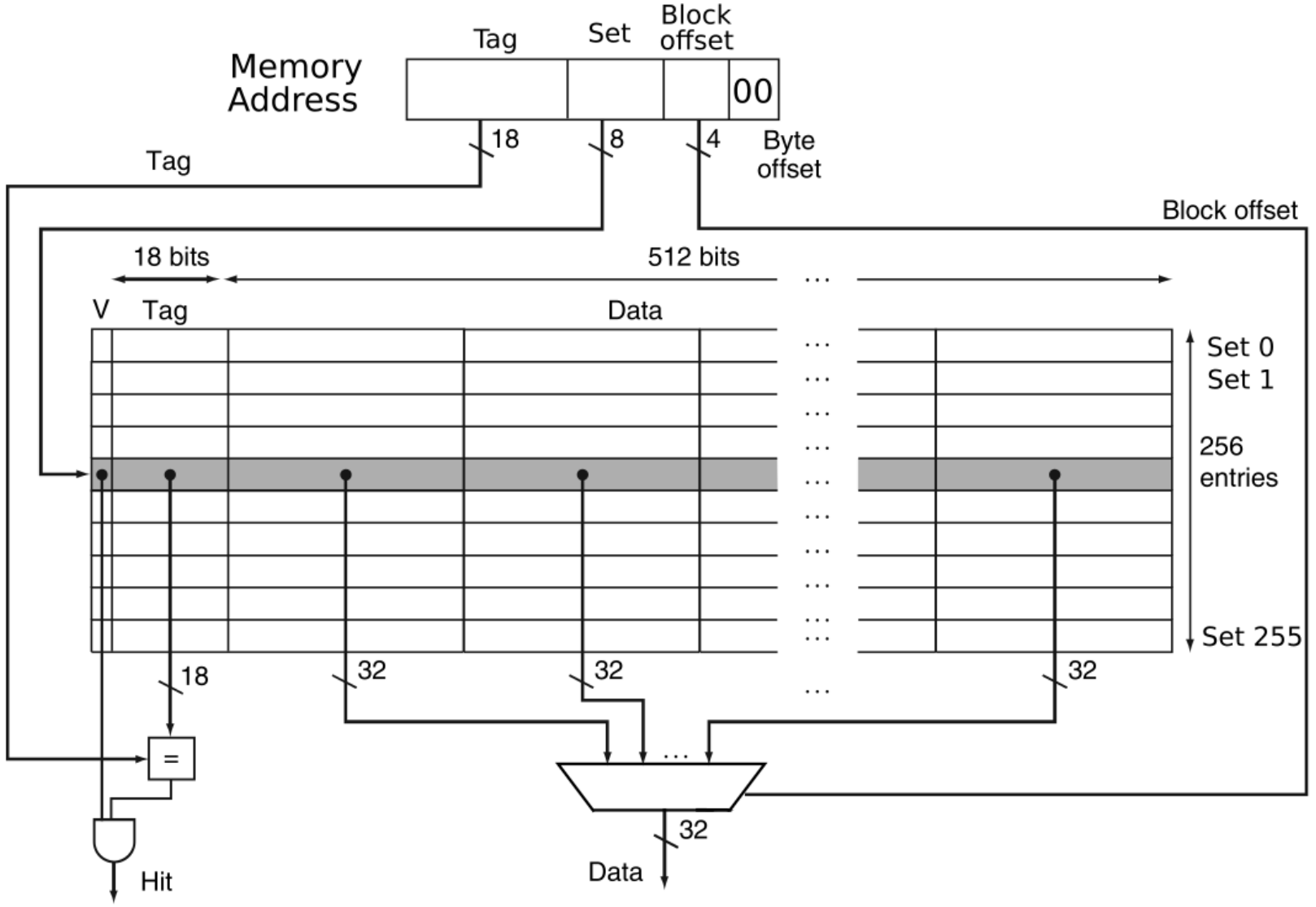


Direct Mapped Cache with b>1

- To take advantage of ***spatial locality***, a cache uses larger blocks containing several consecutive words.
- Let's consider *C= 8* (as before) and *b = 4*
  - *B = C/b = 2* (therefore *S=2*)



256x64 Byte Blocks Direct Mapped Cache

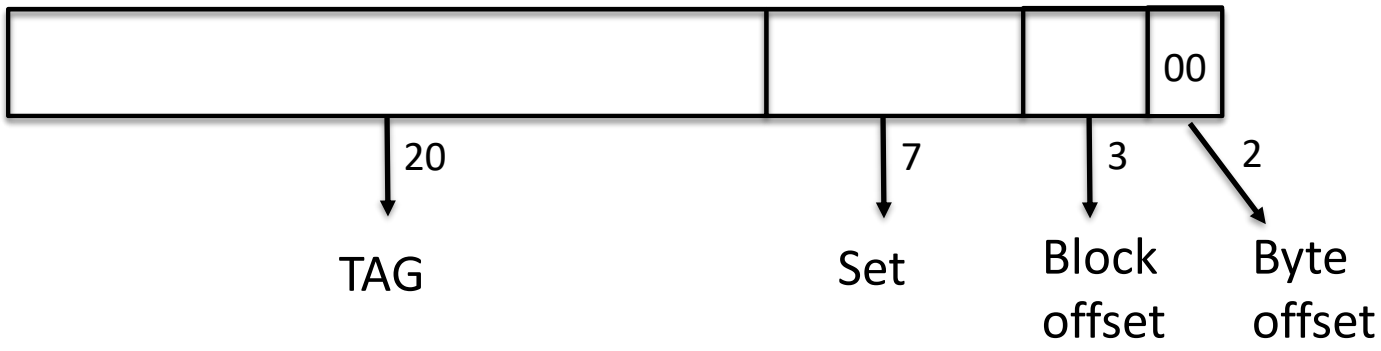


C=16K, S=B=256, b=16

Example1

- Let's suppose 32bit addresses, S=B=128 and b=8 cache. Which is the structure of the address?

- The memory word bytes offset takes *2* bits
- To identify the memory word in a block we need $log_2 8 = 3$ bits (this is the block offset)
- To address all sets, we need $log_2 128 = 7$ bits
- The remaining bits are for the TAG (20 bits)



Example2

- Considering S=B=128, b=8, which is the cache line and the offset within the block that holds the address 0xFEAC?

- 0xFEAC -> 0…01111 1110 1010 1100
- (TAG, IDX, OFF) = (0…01111, 1110101, 01100)
- The cache line has index 117, i.e., the 118th entry
- The offset in the cache block is 3, i.e., the 4th memory word.

TAG	111	110	101	100	011	010	001	000
0..01111	0xFEBC	0xFEB8	0xFEB4	0xFEB0	**0xFEAC**	0xFEA8	0xFEA4	0xFEA0

Example3

- Let's consider the following snippet of C code

```
// A,B,C already allocated and A,B initialized
for(i=0;i<16;i++)   C[i] = A[i]+B[i];
```

- Consider a processor running @2GHz with a *direct-mapped L1 data cache* with C=128, b=8; $t_M = 100$ cycles and $t_{L1} = 4$ cycles
  - 'A' starts at address 0x00000000, 'B' starts at address 0x00000040, and 'C' starts at address 0x00000080
  - After t_M cycles all b words have been loaded into the cache block
- Compute the number of cache faults (cache misses) and the AMAT *considering only load instructions.*

Example3 (cont)

```
// A,B,C already allocated and A,B initialized
for(i=0;i<16;i++)  C[i] = A[i]+B[i];
```

- Since A starts at memory address 0, the first 8 words of A will be loaded in the Set0 (000) while the second 8 words in the Set1 (001) of the L1 cache.
- Since B starts at memory address 64, the first 8 words of B will be loaded in the Set2 (010) while the second 8 words Set3 (011) of the L1 cache.
- Therefore, there are no conflict on the sets, and in total we have 4 misses (2 for the two cache lines containing the 16 words of A, and 2 for B) and 32 cache hits (16+16, load instructions)
- In general, if there is not temporal locality (reuse) the number of misses is $\frac{N}{b}$ where *N* is the number of load instructions (in this example $N = 2 * 16$)
- *AMAT* is defined as *HitTime + MissRate*MissPenalty*, in our case the $MissRate = \frac{4}{32}$, thus $AMAT = \left(4 + \frac{4}{32} * 100\right) * ClockCycleTime,$ where $ClockCycleTime = \frac{1}{2GHz} = 0.5ns,\ \ AMAT = 8.25ns$

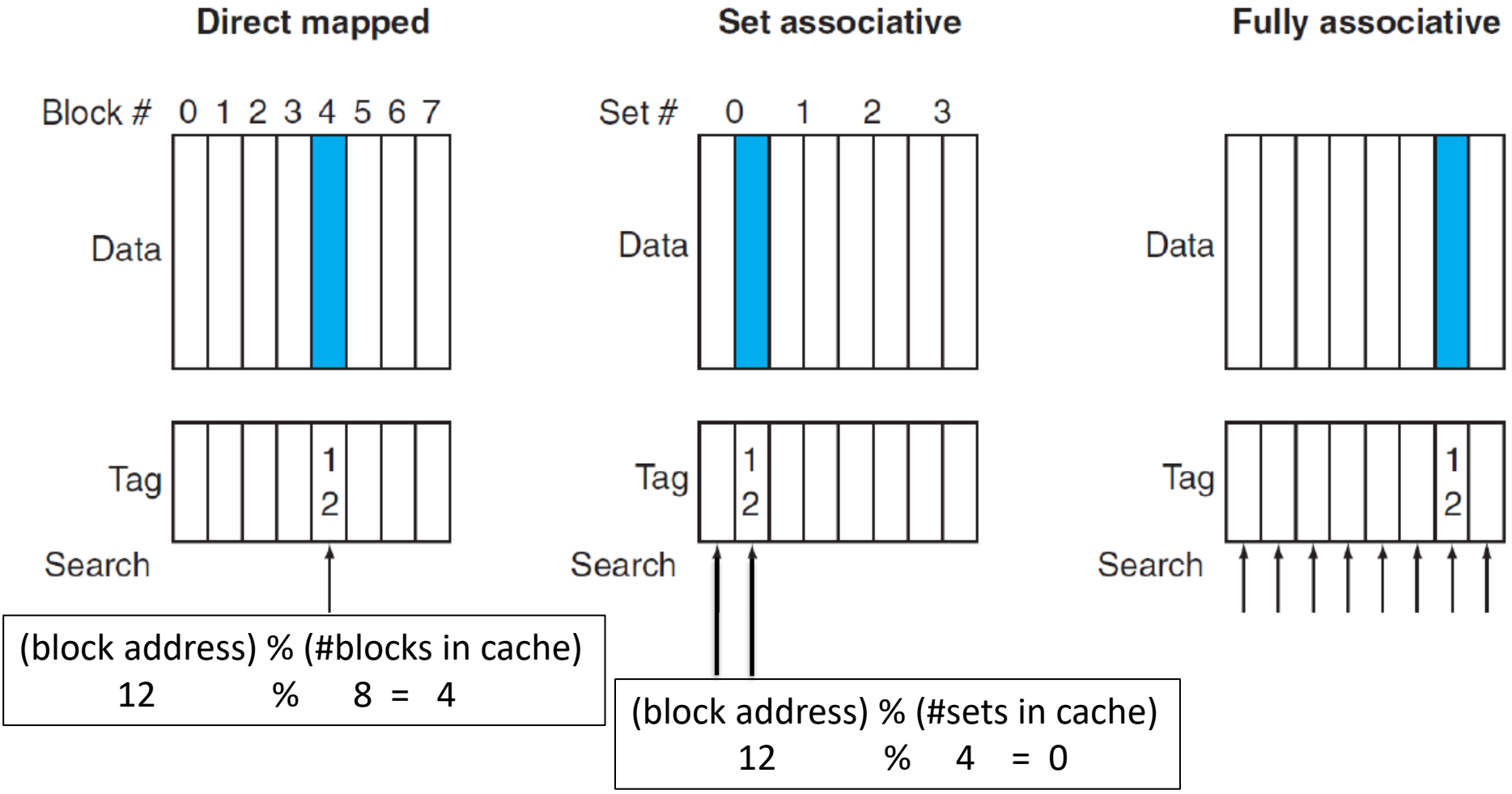
Direct Mapped Cache pros and cons

- **Pros**:
  - Simple to realize
  - Very fast in case of cache hits
- **Cons**:
  - Too rigid, a given memory word can be placed in only one single entry of the cache
    - The substitution of a cache line does not consider *temporal locality*
  - This rigidity *may have a big impact on the number of cache conflicts* (***thrashing***) depending on the memory placement and usage of the data structures
    - E.g., suppose A and B of the previous example mapped in the same set

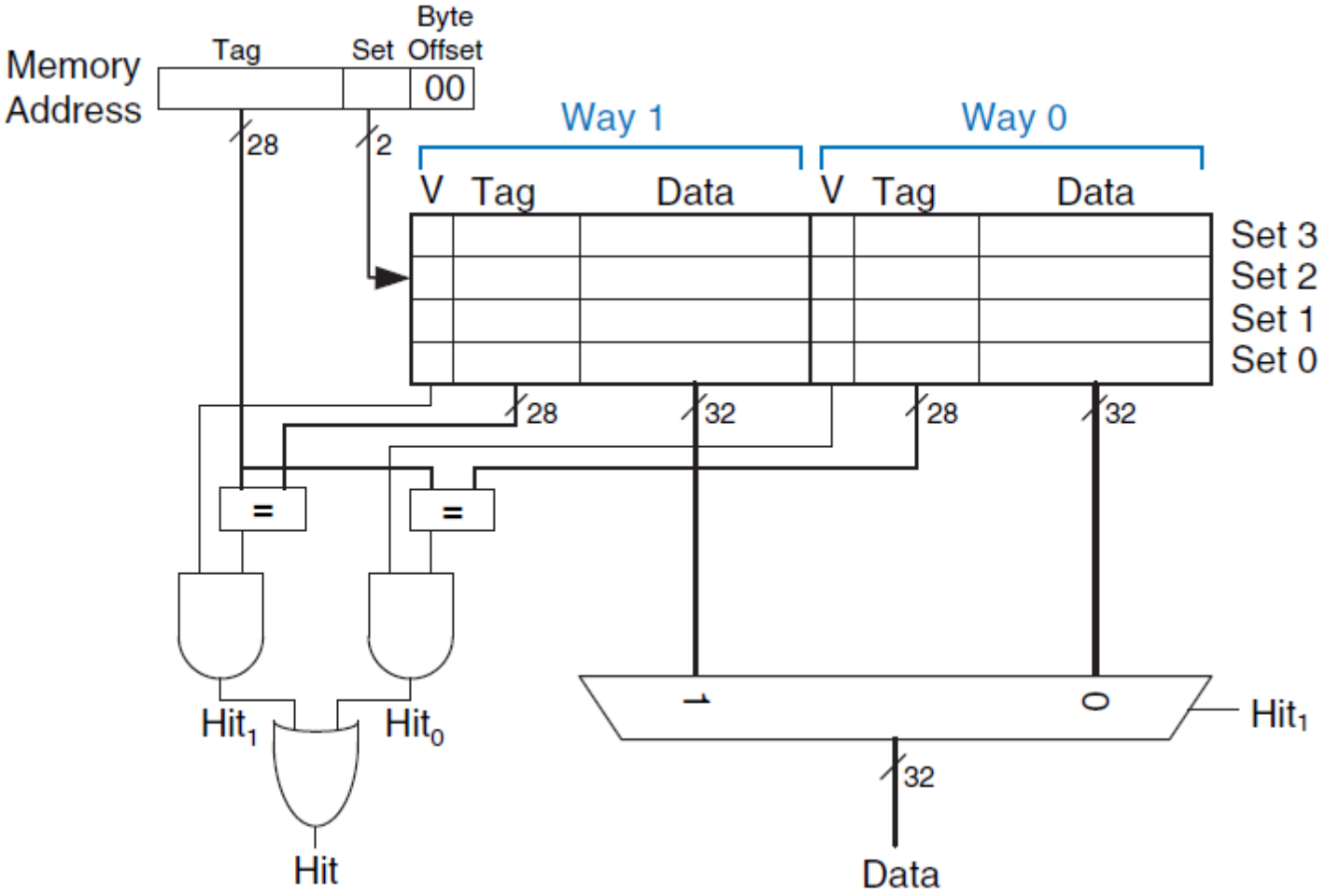
Associative Caches

- **Fully associative cache**
  - Blocks can be placed in any entry in the cache
  - To find a given block, it requires to search in all entries (in parallel)
  - Each entry has a comparator, thus the HW cost significantly increases
- **N-way set-associative cache**
  - Blocks can be placed in a fixed number of N entries (called Set)
  - Each block address is mapped *to exactly one set* (as in the direct mapped cache where #S=#B), which contains N entries
  - A block can be placed in any entry of the set (as in the fully associative cache)
  - To find a given block, the search is made in parallel in the N entries of the set (using *N* distinct comparators)

Associative Caches

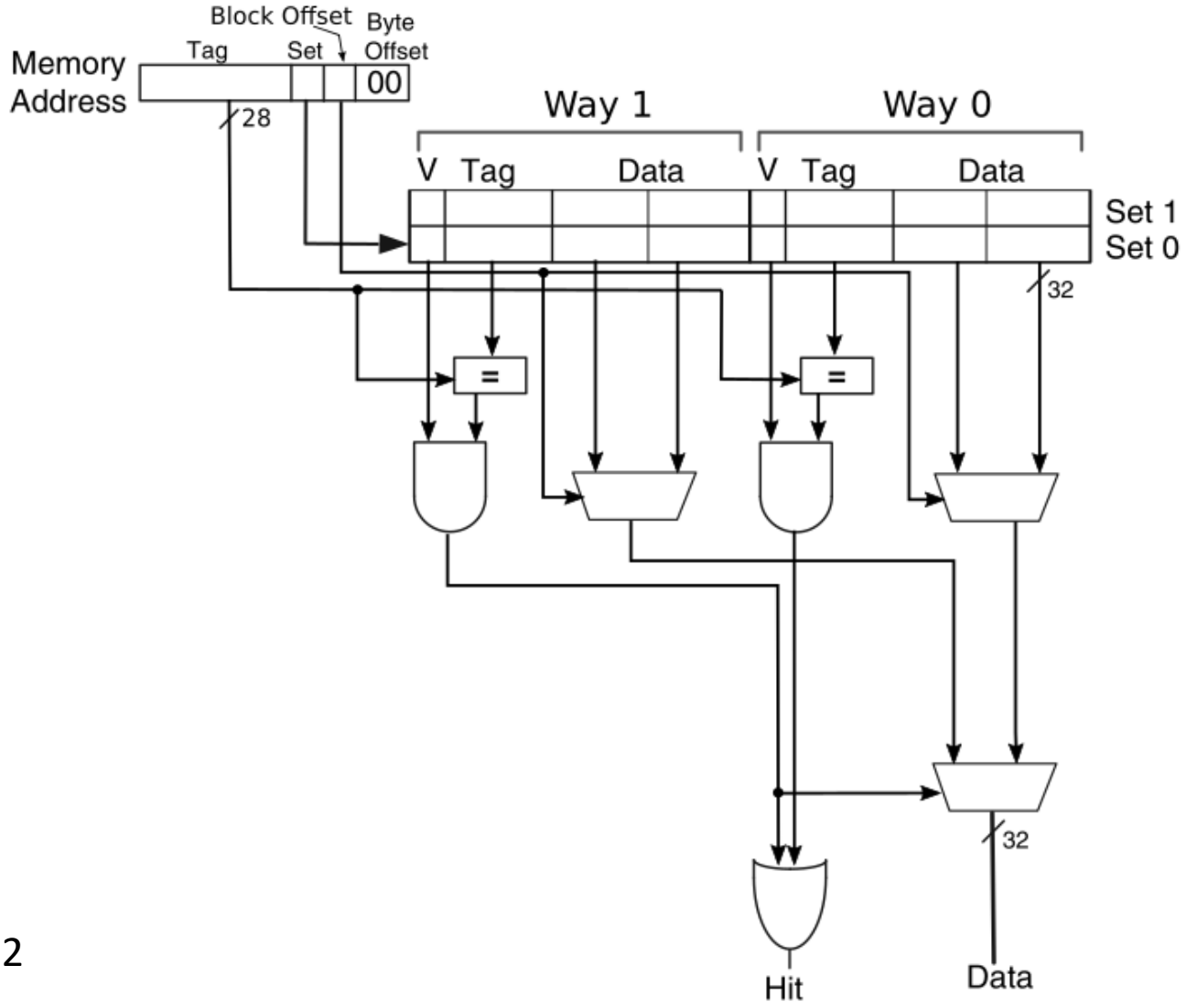


2-way Set-Associative Cache (b=1)



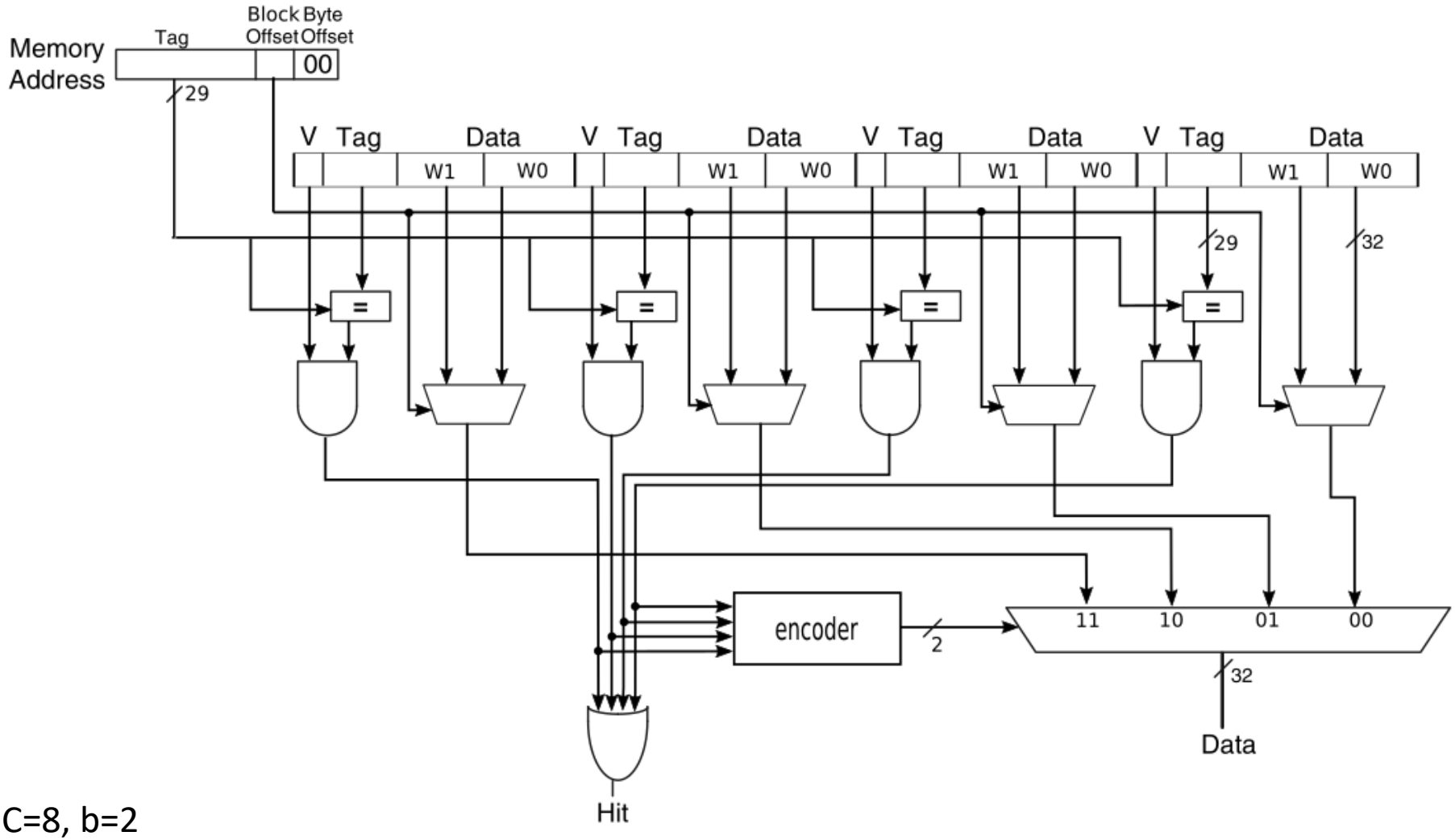
C=8, S=4, b=1

2-way Set-Associative Cache (b=2)



C=8, S=4, b=2

Fully Associative Cache (b=2)



C=8, b=2

Cache organization summary

- For a given capacity C, we must decide the block size *b*, the number of blocks *B* (i.e., cache lines, *C/b*) and the number of ways (thus the number of blocks in a Set)

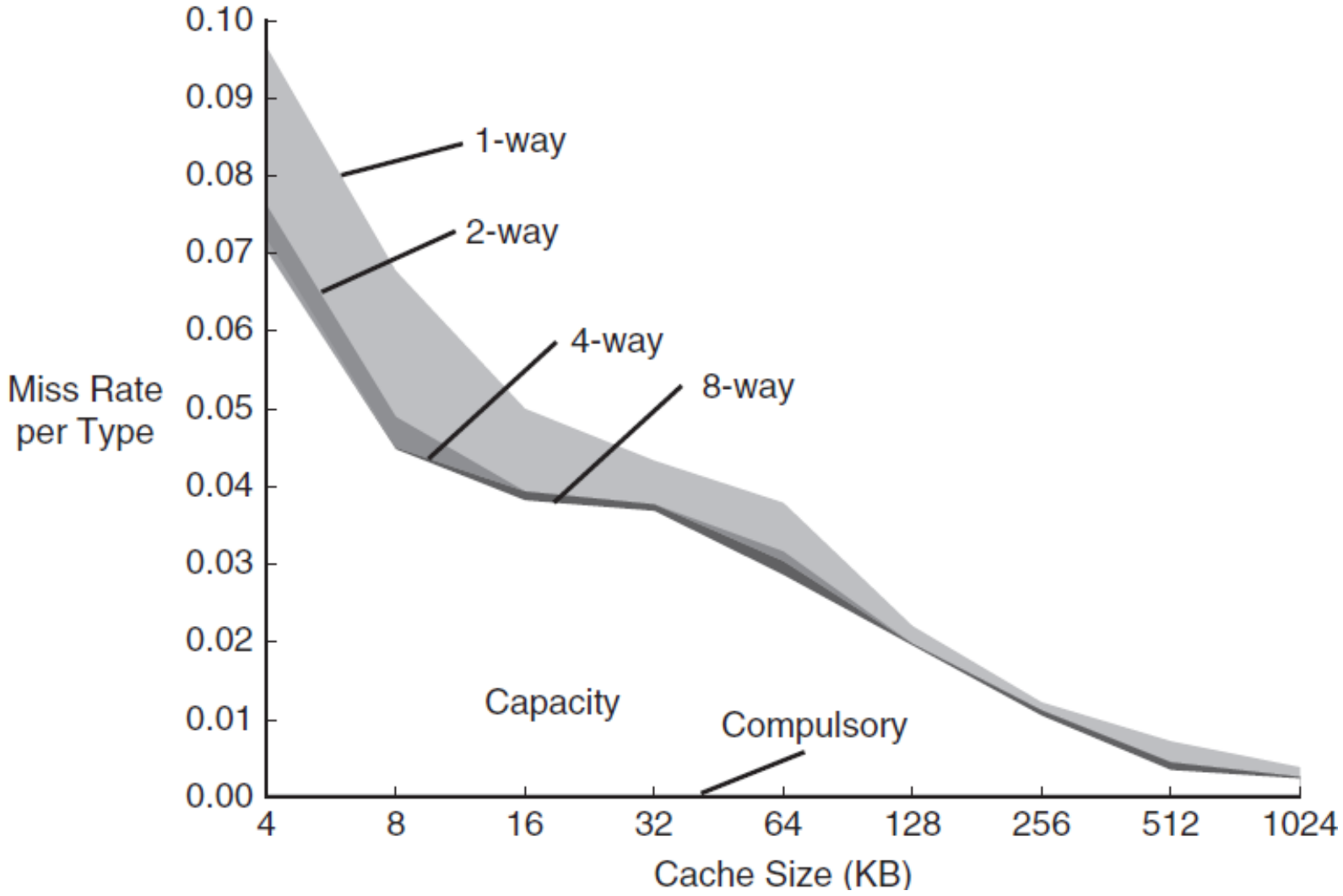
Organization	Number of Ways (N)	Number of Sets (S)
Direct Mapped	1	B
Set Associative	$1 < N < B$	B/N
Fully Associative	B	1

- Cache misses can be reduced by increasing the capacity *C*, the block size *b* and the associativity *N*
  - We are interested in to evaluate the impact of *b* and *N* **keeping *C* fixed**

Kinds of cache misses

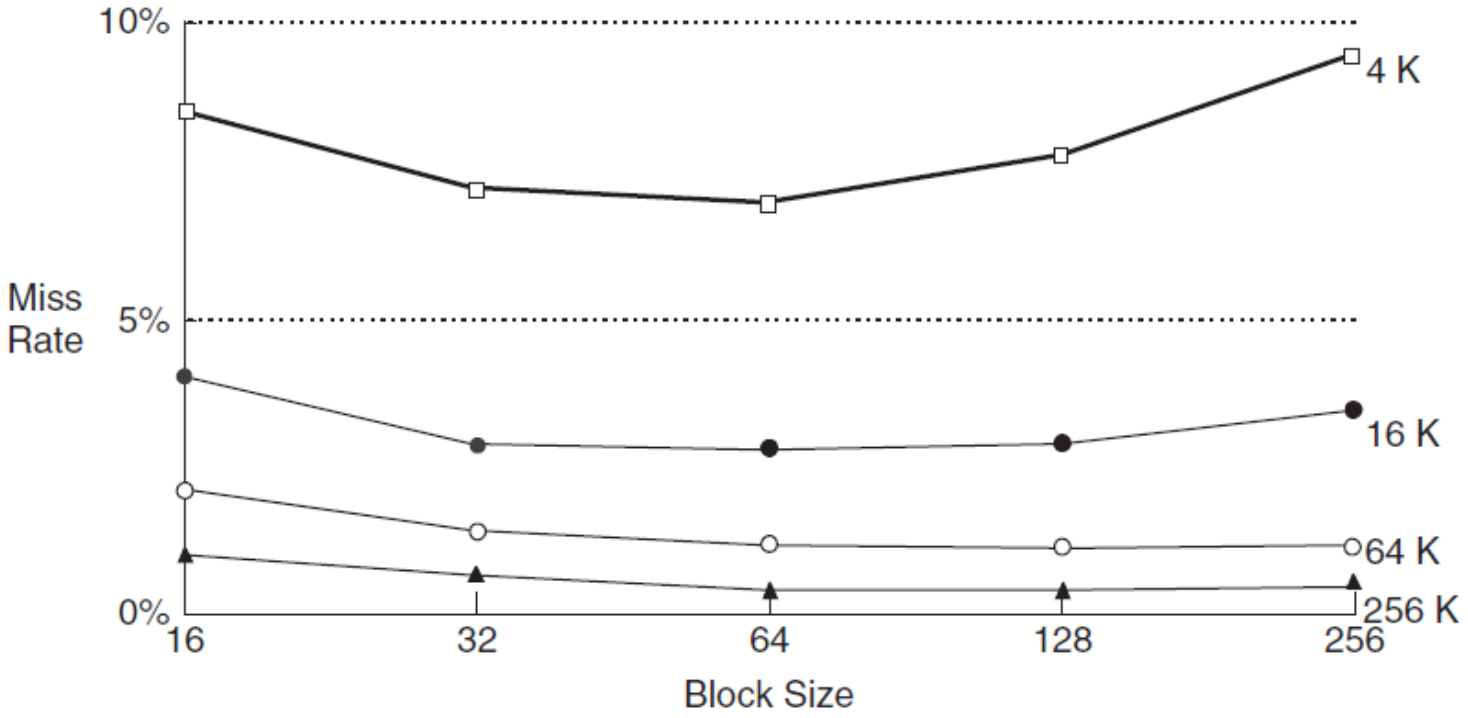
- **Compulsory misses**
  - These are cache misses caused by the first access to a block that has never been in the cache.
- **Capacity misses**
  - These are cache misses caused when the cache cannot contain all the blocks needed. Some blocks are evicted and later retrieved.
- **Conflict misses**
  - Only for direct mapped and set-associative caches
  - These are cache misses that are eliminated in a fully associative cache of the same size

Miss rate vs cache size and associativity



Based on SPEC2000 benchmark

Miss rate vs block size and cache size



- As block size increases, there are more opportunity to exploit spatial locality, but
  - The miss penalty increases (more time to load a block)
  - Increases the probability of conflict misses because the number of sets decreases

Reducing Miss Rate Summary

- Increase the block size
  - Improve spatial locality but increase the miss penalty
- Increase the associativity
  - Less conflicts but higher hit time
- Increase the case size
  - Less capacity miss and conflicts but higher hit time
- Cache-oblivious algorithm (not yet examined)
  - Impact on the complexity of compilers
  - Impact on programmers

Handling cache misses

- A cache miss *stalls the entire processor*, freezing the contents of all registers while waiting for memory
  - Indeed, more advanced processor allows ***out-of-order execution*** of other instructions while waiting for memory
- The steps taken for a *cache miss* are (***fault management***):
  1. Tell the next memory level to *read* the missing value
  2. Wait for the memory to respond (this can take multiple cycles)
  3. Update the corresponding cache line with the data received
  4. Restart the instruction execution (now it is a cache hit)

Handling writes

- **Write hits (same TAG)**
  - If the store instruction writes the data only into the cache, then cache and memory would have different values (cache and memory are **inconsistent** – the main memory contains stale data)
  - We need to define a strategy for write hits; two options:
    1) **Write-Through** 2) **Write-Back**
- **Write misses (different TAG)**
  - Should we fetch the block from memory to cache and then overwrite the missing word?
    - This policy is called **write-allocate**. The block is loaded into the cache followed by a write hit (mainly used in the ***Write-Back*** policy)
  - Should we write the word directly to the next memory level?
    - This policy is called **no-write-allocate**, the block is not loaded into the cache (mainly used in the ***Write-Through*** policy)

Write-Through

- Write hits always update both the cache and the next memory level
- Pros
  - Simple solution, easy to implement
  - Data is ***always consistent*** between the two memory levels
- Cons
  - The speed of writes depends on the write speed of the lower memory level
  - Higher memory traffic, for every single write there may be multiples writes in each memory level

Write-Back

- Write hits update only the cache, then the modified block is written to the lower memory level when it is replaced
- Pros
  - The speed of writes is that of the cache
  - Lower memory traffic w.r.t. Write-Through, subsequent writes to the same cache block do not produce traffic with the lower memory level
- Cons
  - We need to keep track of modified blocks (**dirty bit**)
  - More complex to implement than Write-Through
  - The cache line replacement is more costly

Write-Through vs Write-Back example

Example 8.12 WRITE-THROUGH VERSUS WRITE-BACK

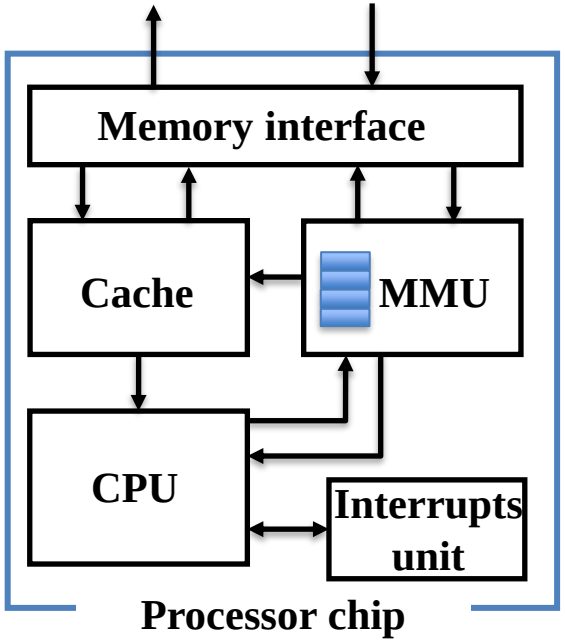
Suppose a cache has a block size of four words. How many main memory accesses are required by the following code when using each write policy: write-through or write-back?

```
MOV R5, #0
STR R1, [R5]
STR R2, [R5, #12]
STR R3, [R5, #8]
STR R4, [R5, #4]
```

Solution: All four store instructions write to the same cache block. With a write-through cache, each store instruction writes a word to main memory, requiring four main memory writes. A write-back policy requires only one main memory access, when the dirty cache block is evicted.

Optimizing writes

- Since writing to off-chip memory is costly, memory stores are buffered
- A write buffer is used to hold the data waiting to be written to memory
- Execution continues immediately after writing the data into the cache **and** into the write buffer
  - Memory stores to main memory are executed in parallel with the CPU computation
- The CPU stalls only if the write buffer is full, therefore the main memory requested bandwidth is a critical factor, *particularly for the Writhe-Through* cache model!



Write-Back Considerations

- Write-Back can improve performance especially when the CPU generates store instructions faster than what the main memory can handle
- However, the cost of cache writes is higher if a write miss occurs, we first must write the block back to memory (if the dirty bit is 1).
  - This requires at least two cycles even for a write hit: a cycle to check for a hit followed by a cycle to actually perform the write
  - *Alternatively*, we may use a **write buffer** to temporarily hold the data to write while the cache block is checked for a hit. The processor does the cache lookup and places the data in the write buffer during the normal cache access cycle. Assuming a cache hit, the new data is written from the write buffer into the cache on the next unused cache access cycle (*pipelining of accesses*)

Cache Replacement

- In a *direct mapped* cache, the requested block can go in exactly one position, thus we have no choice
  - If the block to be replaced has been modified and the write policy is Write-Back, we must update the lower-level memory
- In an associative cache, we have a choice of where to place the requested block
  - *Fully associative* cache, all blocks are candidates for replacement
  - *N-way set-associative* cache, we must choose among the *N* blocks in the selected set

Cache Replacement Policy

- Which replacement policy to adopt?
- The most used scheme is **Least Recently Used** (**LRU**)
  - It considers temporal locality, the block replaced is the one that has been ***unused*** for the longest time
  - For a 2-way set-associative cache, it can be implemented with 1 bit (*use bit -- U*), for a 4-way is still doable with 2 bits, for more that 4-ways it becomes quite complicated (*pseudo-LRU*)
- For high associative caches, a **random** policy gives approximately the same performance as LRU

(*) We will discuss cache replacement policies in more details when we will study virtual memory

Example

- Consider a small cache with 4 blocks, b=1. Find the number of misses for a **direct-mapped** cache for the following ordered sequence of block addresses 0, 8, 0, 6, 8.

Block address	Cache block
0	(0 modulo 4) = 0
6	(6 modulo 4) = 2
8	(8 modulo 4) = 0

Address of memory block accessed	Hit or miss	Contents of cache blocks after reference			
		0	1	2	3
0	miss	Memory[0]			
8	miss	Memory[8]			
0	miss	Memory[0]			
6	miss	Memory[0]		Memory[6]	
8	miss	Memory[8]		Memory[6]	

Example

- Consider a small cache with 4 blocks, b=1. Find the number of misses for a **2-way set-associative** cache for the following ordered sequence of block addresses 0, 8, 0, 6, 8 (**LRU replacement policy**).

Block address	Cache set
0	(0 modulo 2) = 0
6	(6 modulo 2) = 0
8	(8 modulo 2) = 0

Address of memory block accessed	Hit or miss	Contents of cache blocks after reference			
		Set 0	Set 0	Set 1	Set 1
0	miss	Memory[0]			
8	miss	Memory[0]	Memory[8]		
0	hit	Memory[0]	Memory[8]		
6	miss	Memory[0]	Memory[6]		
8	miss	Memory[8]	Memory[6]		

Example

- Consider a small cache with 4 blocks, b=1. Find the number of misses for a **fully associative** cache for the following ordered sequence of block addresses 0, 8, 0, 6, 8.

Address of memory block accessed	Hit or miss	Contents of cache blocks after reference			
		Block 0	Block 1	Block 2	Block 3
0	miss	Memory[0]			
8	miss	Memory[0]	Memory[8]		
0	hit	Memory[0]	Memory[8]		
6	miss	Memory[0]	Memory[8]	Memory[6]	
8	hit	Memory[0]	Memory[8]	Memory[6]	

Improving processor performance

- Let's consider again the cache performance example
- What happens if the processor is made faster with the same memory system?
- Suppose we speed up the clock rate by a factor of 2 (e.g., 2 -> 4 GHz)

$$CPI_{Stall-Instr} = 1 * 0.02 * 200 = 4$$
$$CPI_{Stall-Data} = 0.36 * 0.04 * 200 = 2.88$$
$$CPI_{Stall.} = 4 + 2.88 = 6.88$$
$$CPI = 2 + 6.88 = 8.88$$

- The fraction of time spent on memory stalls rises from 63% to 77%
  - 3.44/5.44 = 0.63 → 6.88/8.88 = 0.77
- The execution time improves of a factor 1.23:

$$\frac{CPU_{time1}}{CPU_{time2}} = \frac{IC * CPI_{Stall1} * ClockCycleTime}{IC * CPI_{Stall2} * \frac{ClockCycleTime}{2}} = \frac{5.44}{\frac{8.88}{2}} = 1.23$$

Improving processor performance

- The same happens if we improve the processor architectures by a factor of 2 (i.e., the CPI decreases from 2 to 1)
  - for example, because we improved the CPU pipeline organization
- In both situations (i.e., increasing clock rate and decreasing CPI), *the time spent on memory stalls takes an increasing fraction of the total execution time*, which produces a reduced impact on the overall performance (1.23) compared to the speed up factor of 2
- In other words, the relative efficiency of these optimizations is low:

$$\varepsilon = \frac{1.23}{2} = 61\%$$

- To face this issue almost all modern systems, support *multiple levels of caching,* which ***helps reducing the miss penalty factor***

Multi-level caches in modern processors

Characteristic	ARM Cortex-A8	Intel Nehalem
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	32 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	4-way (I), 4-way (D) set associative	4-way (I), 8-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, Write-allocate(?)	Write-back, No-write-allocate
L1 hit time (load-use)	1 clock cycle	4 clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 1 MiB	256 KiB (0.25 MiB)
L2 cache associativity	8-way set associative	8-way set associative
L2 replacement	Random(?)	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate (?)	Write-back, Write-allocate
L2 hit time	11 clock cycles	10 clock cycles
L3 cache organization	–	Unified (instruction and data)
L3 cache size	–	8 MiB, shared
L3 cache associativity	–	16-way set associative
L3 replacement	–	Approximated LRU
L3 block size	–	64 bytes
L3 write policy	–	Write-back, Write-allocate
L3 hit time	–	35 clock cycles

Multi-level caches

- The second-level (L2) cache is accessed whenever a miss occurs in the first-level (L1) cache.
- If the L2 cache contains the desired data, the miss penalty for the L1 cache will be essentially the access time of the L2 cache, which will be much less than the access time of main memory
  - different technologies for off-chip memories
- The same happens for a third-level (L3) cache, if it exists.
- Only if L1, L2 and L3 do not contain the data, a main memory access is required, and a larger miss penalty is paid

Multi-level cache and *AMAT*

- Quantitative example:
  - $t_M = 50ns$ (main memory service time), $t_{L1} = 1ns\ t_{L2} = 6ns\ t_{L3} = 10ns$
  - Miss rates: 10%, 1.5% and 0.4% for L1,L2, and L3, respectively

1. No cache: *AMAT*= 50ns
2. Only L1 cache: *AMAT*= 1+0.1*50= 6ns
3. L1 and L2 caches: *AMAT*= 1+0.1*(6+0.015*50)= 1.675ns
4. L1, L2 and L3 caches: *AMAT*= 1+0.1*(6+0.015*(10+0.004*50))= 1.6153ns

Multi-level cache and CPI

- Remember that

$$CPI_{Stall} = Miss\ rate\ memory\ instr. * Miss\ penalty$$

- With multilevel caches, memory-stall clock cycles can be defined as the sum of the stall cycles coming from all cache levels
  - Let's assume that loads and stores have the same miss rates and penalties

$$CPI_{Stall} = MissRate_{L1} * MissPenalty_{L1} + GlobalMissRate_{L2} * MissPenalty_{L2} + GlobalMissRate_{L3} * MissPenalty_{L3} + \ldots$$

where, for a given level LN (N>1):

$$GlobalMissRate_{LN} = MissRate_{L1} * MissRate_{L2} * \ldots * MissRate_{LN}$$

Multi-level cache and CPI

- Suppose a 2-level cache system with:
  - $MissRate_{L1} = 2\%, \quad MissRate_{L2} = 20\%$
  - L2 cache access time 20 cycles, main memory access time 200 cycles

compute a) the $GlobalMissRate_{L2}$ and b) the CPI_{Stall}.

a) $\text{GlobalMissRate}_{\text{L2}} = \text{MissRate}_{\text{L1}} * \text{MissRate}_{\text{L2}} = 0.02*0.2 = 0.04 \ (4\%)$

b) A miss in the L1 cache can be satisfied either by L2 cache or by the main memory. The miss penalty is 20 cycles if we have an hit in L2, and 200 cycles if we have a miss in L2, thus:

$$MissPenalty = \ 20 + 0.2 * 200 = 60 \ cycles$$

Therefore, $CPI_{Stall} = \ MissRate_{L1} \ * MissPenalty = 0.02 * 60 = 1.2 \ cycles$

(in a different way)

b) By considering $MissPenalty_{L1} = 20$ and $MissPenalty_{L2} = 200$ and using the previous formula (previous slide), we have:

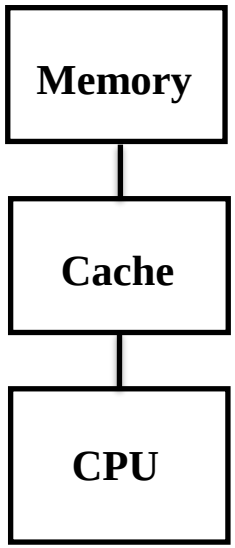
$$CPI_{Stall} = 0.02 \ * 20 + (0.02 * 0.2) * 200 = 1.2 \ cycles$$

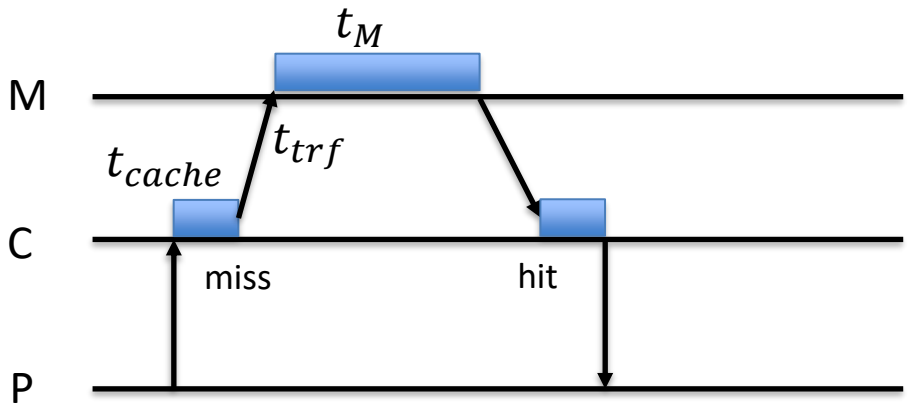
Designing the Memory System

- The *miss penalty* can be reduced increasing the bus bandwidth between DRAM and cache
- Three possible organizations:
  - **Simple**: one-word at a time is read from memory
  - **Wide-memory**: N words at a time are read from memory
  - **Interleaved**: K independent memory banks capable to serve K requests simultaneously

- Let's consider the cache block transfer time for *Simple* vs *Interleaved* memory organization

Cache blocks transfer





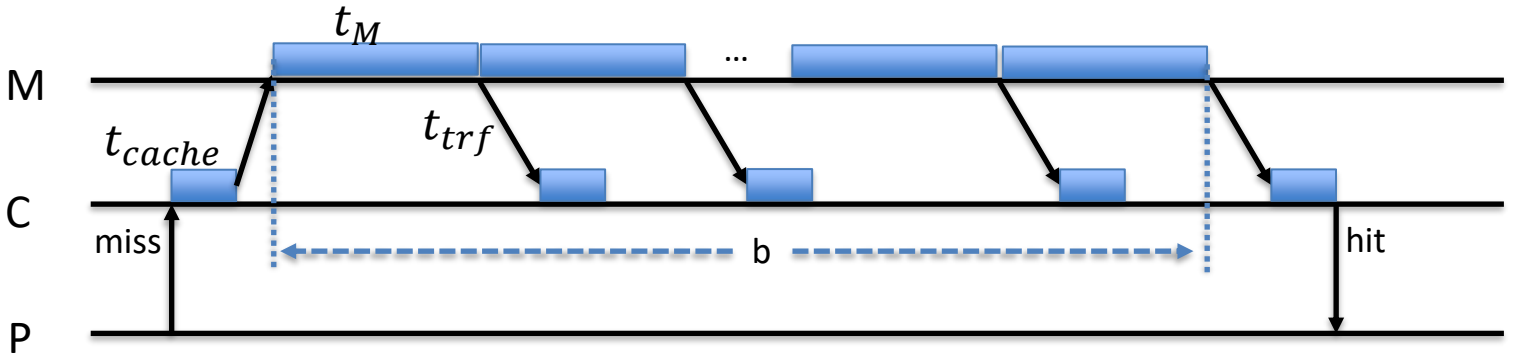
- Suppose a single main memory module
- The *Memory* and *Cache* service time is t_M, t_{cache}, respectively
- The offered memory bandwidth is $B_M = \frac{1}{t_M}$ and b=1
- The memory access time as seen by the CPU is about

$$t_a = 2\,t_{trf} + 2\,t_{cache} + t_M$$

Cache blocks transfer

- If the cache line holds *b>1* memory words, the cost to transfer the entire block in case of a cache miss is (*cache fault management*):

$$t_{miss} = 2t_{trf} + 2t_{cache} + b\ t_M \approx b\ t_M$$

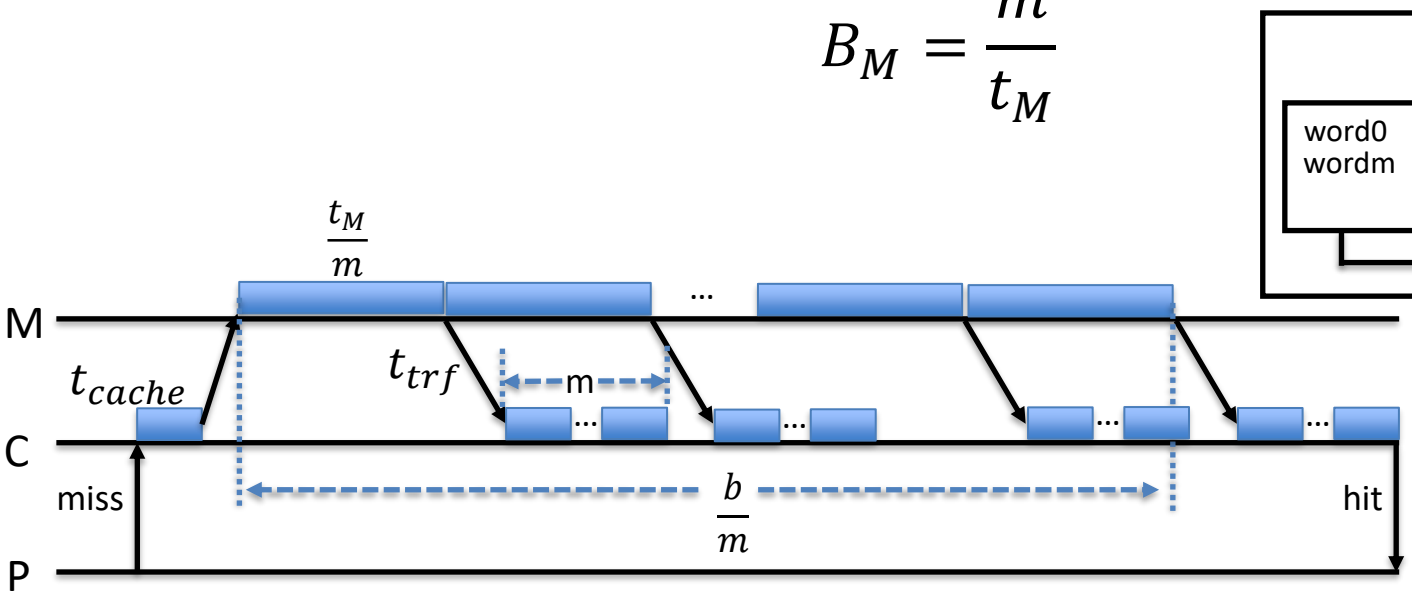


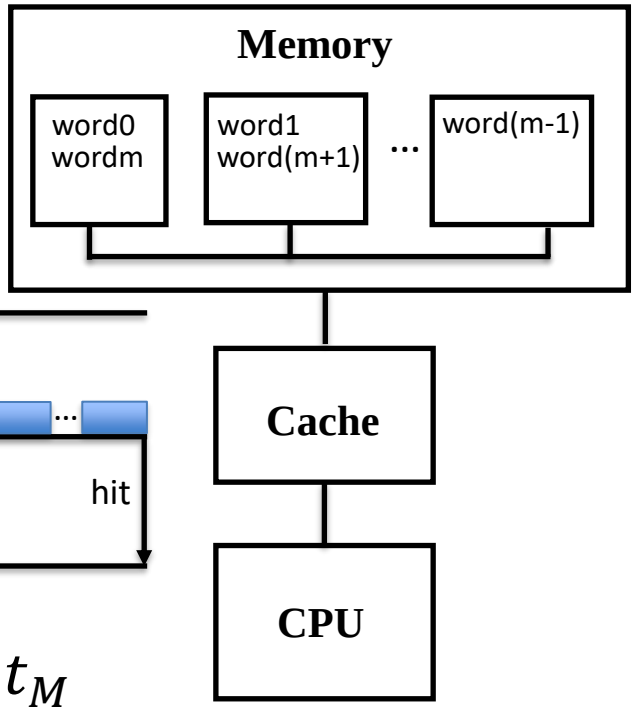
- Considering $b = 8$ and $t_M = 80$, t_{cache}= 1 and $t_{trf} = 6$ clock cycles, the cost of the cache miss due to the transferring of all data in the block is very high (>650 clock cycles)
  - The CPU stalls for several hundred cycles!

Cache blocks transfer

- By using modular memory (*memory interleaving*) of *m* modules we increase the offered bandwidth by a factor of m:

$$B_M = \frac{m}{t_M}$$





$$t_{miss} = 2t_{trf} + (\sim 1 + 1)\, t_{cache} + \frac{b}{m}\, t_M$$

- If we set $m = b$ we have again $t_{miss} \approx t_M$

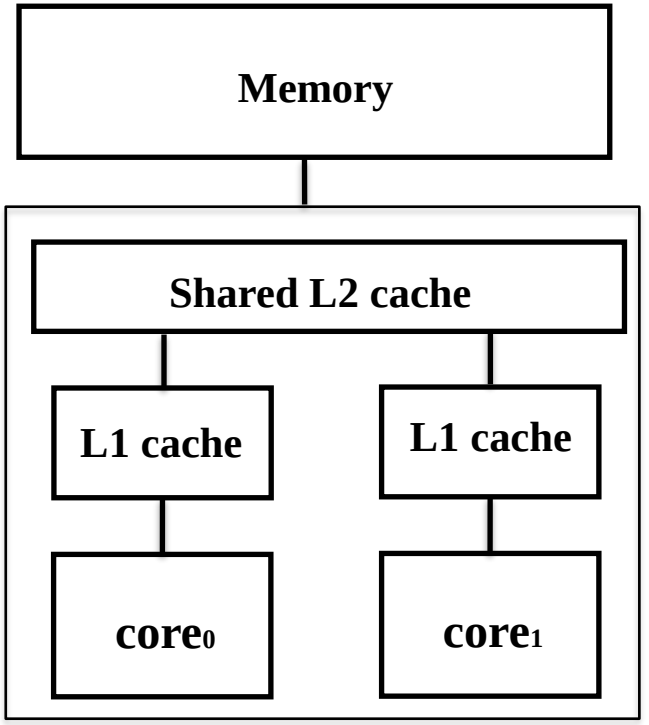
~1 means that we can write m words into a cache line in a clock cycle.

Cache issues

- Caching is essential to reduce the von Neuman bottleneck and achieve reasonable performance on modern systems
- However, caching introduces some problems in ***multiprocessor/multicore systems***
  - **Cache coherence problem**
  - *False sharing* performance degradation in cache-coherent multiprocessors
    - Two unrelated variables are placed in the same cache block and accessed in read/write mode by different threads!

Cache Coherence Problem (brief note)

- Let's suppose a SMP architecture
  - SMP: Symmetric Multiprocessors
- Caching of both private and shared data
- Private core data is cached in L1, thus reducing the AMAT as well as off-chip memory communications.
- When shared data are cached, the shared values may be replicated in multiple private core caches.
  - This also reduce memory contention!
- What happens if shared data are written?

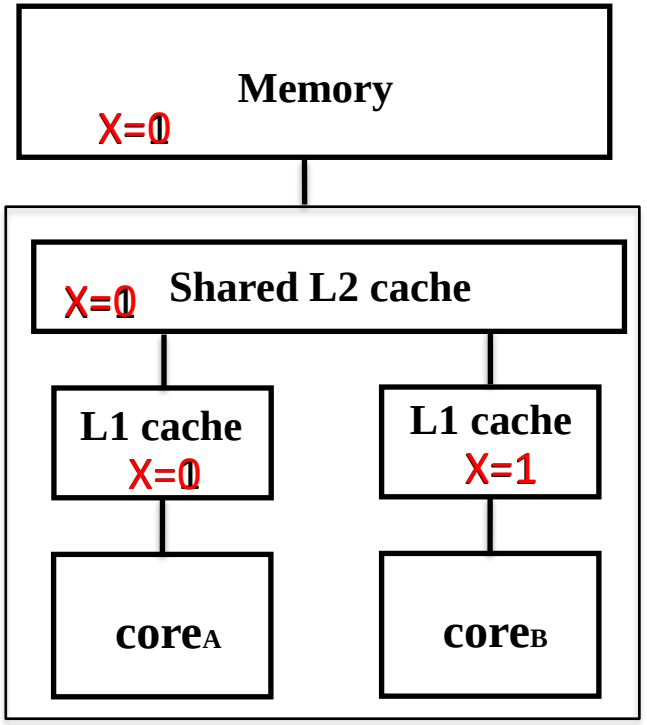


Caching of shared data introduces a new problem: cache coherence

Cache Coherence Problem (brief note)

Write-through caches

Time	Event	L1-core1	L1-core2	M
0				X=1
1	A reads X	X=1		X=1
2	B reads X	X=1	X=1	X=1
3	A writes X	X=0	X=1	X=0



Write invalidate protocol (brief note)

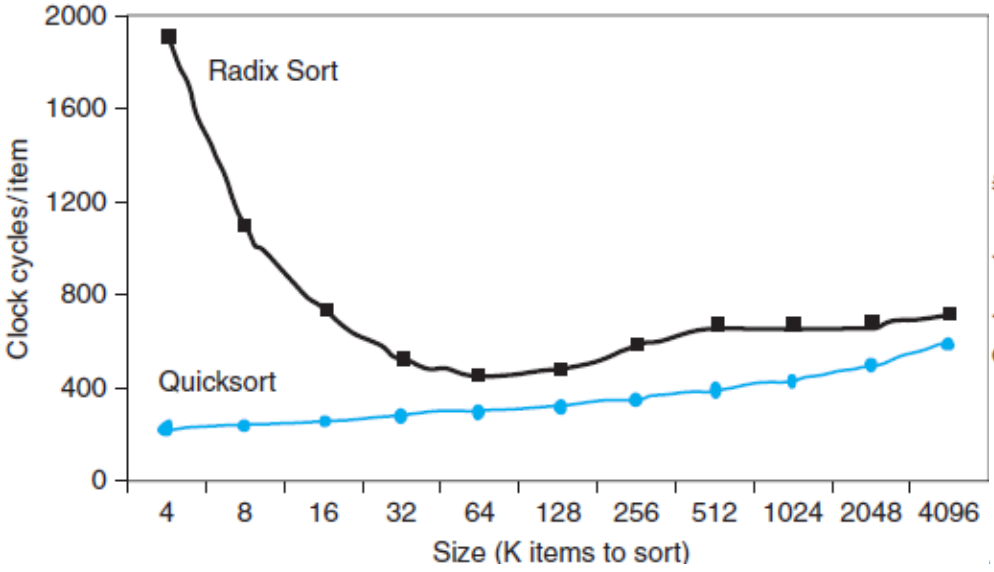
- The *write invalidate protocol* invalidates copies of the data present in other caches on a write operation
- A simple implementation uses a *snooping bus protocol* for ensuring the processor acquires exclusive access to a cache block before it writes into it

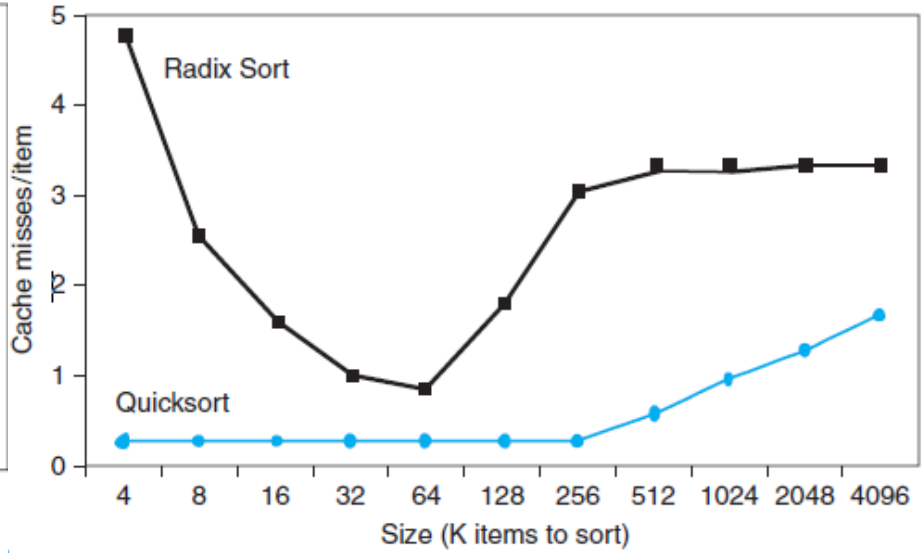
Write-back caches

Event	Bus activity	L1-core1	L1-core2	M
				X=1
A reads X	Miss for X	X=1		X=1
B reads X	Miss for X	X=1	X=1	X=1
A writes X	Invalidation for X	X=0	not valid	X=1
B reads X	Miss for X	X=0	X=0	X=0

Cache Interaction with Software

- Cache misses depend on memory access patterns
  - Algorithm behavior
  - Compiler/Programmer **optimizations** for memory access





- *False sharing* can be another important issues in multiprocessors with automatic cache-coherence

Software Optimizations

- **Goal**: maximize cached data reuse by enhancing temporal and/or spatial locality
- Two well-known techniques to reduce data miss rate are: *loop interchange* and data blocking
- Example: loop interchange (for row-major ordering – C)

Before

```
for (j=0;j<100;++j)
  for(i=0;i<1000;++i)
      A[i][j] = 2*A[i][j];
```

After

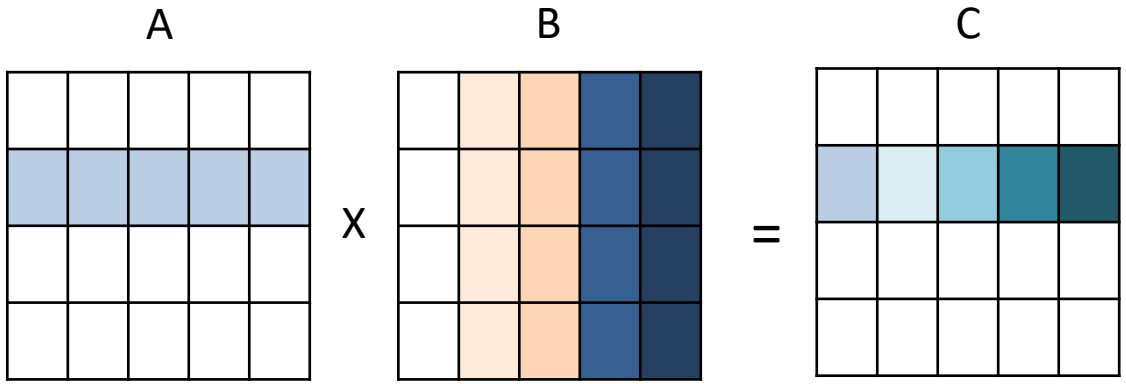
```
for (i=0; i<1000; ++i)
  for(j=0; j<100; ++j)
      A[i][j] = 2*A[i][j];
```

Software Optimization via blocking

- Loop interchange improves spatial locality
- Blocking improves temporal locality
- Example: Matrix Multiplication (GEMM algorithm)

Before (textbook algorithm)

```
for (i=0;j<N;++i)
 for(j=0;j<N;++j) {
     c = 0;
     for(k=0;k<N;++k)
       c+= A[i][k] * B[k][j];
     C[i][j] = c;
 }
```

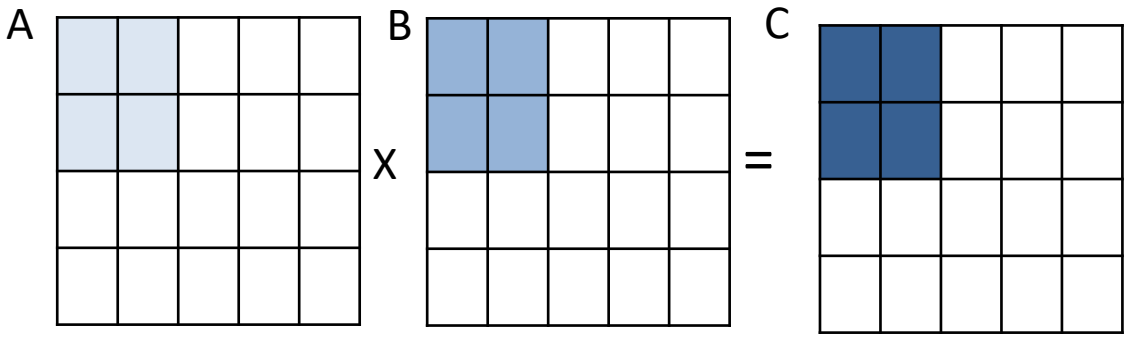


Software Optimization via blocking

- Blocking version of GEMM with block size B (*block factor*)

After

```
for (jj=0; jj<N; jj+=B)
  for(kk=0; kk<N; kk+=B)
    for(i=0;i<N;++i)
      for(j=jj; j < min(jj+B,N); ++j) {
        for(c=0,k=kk; k<min(kk+B,N); ++k)
          c+= A[i][k] * B[k][j];
        C[i][j] += c;
      }
```



block size BxB (2x2)